

SQL e Dintorni

PAUSA 12:50 fino alle 13:00

TEST dalle 10:05 Durata 60 minuti min / 85 max

SQL (Structured Query Language)



Structured Query Language (SQL) è un linguaggio di interrogazione (query) utilizzato per creare, modificare e gestire i dati in un database relazionale.

Si tratta di un linguaggio specifico di dominio (DSL) usato per comunicare con i sistemi di gestione di database relazionali (RDBMS).

SQL: COSA CI SERVE

Useremo un DBMS(database management system) ovvero un sistema software progettato per consentire la creazione, la manipolazione e l'interrogazione efficiente di database, per questo detto anche "gestore o motore del database".

Nello specifico useremo MySQL che si basa sul modello relazionale quindi un RDBMS(relational database management system).

Cosa faremo :

- **Scaricheremo e istalleremo MySQL**
- **Andremo a vedere alcune connotazioni basilari di MySQL**

INSTALLIAMO MySQL

Andiamo ad installare MySQL community (non la versione web) al seguente link:

https://dev.mysql.com/downloads/installer/



(non è obbligatorio creare un account per l'installazione)

INSTALLIAMO MySQL

Proseguiamo avviando l'installer scaricato, eseguiamo i seguenti passaggi:

1. **selezioniamo Developer Default (per semplicità), andiamo avanti**
2. **risolviamo le richieste manuali, facciamo il check e andiamo avanti**
3. **facciamo execute per avvia l'installazione, una volta finita l'installazione andiamo avanti**
4. **passiamo alla configurazione, rimaniamo tutto default e andiamo avanti fino all'inserimento della password scegliendo una password semplice come "root" per semplificarci i passaggi durante il corso**
5. **continuiamo ad andare avanti nella configurazione, se non si vuole far avviare i servizi di MySQL all'avvio di windows allora bisogna rimuovere la spunta su "start the MySQL server at system startup" in windows service**
6. **continuiamo a rimanere le configurazioni di default fino all'ultimo step, facciamo execute per applicare la configurazione di MySQL Server.**
7. **andiamo avanti rimanendo i valori di default fino alla configurazione del server example dove dobbiamo inserire la password scelta prima e fare il check, andiamo avanti fino all'execute**
8. **andiamo a concludere l'installazione proseguendo fino all'ultimo step avviando la shell e la workbench.**

MySQL



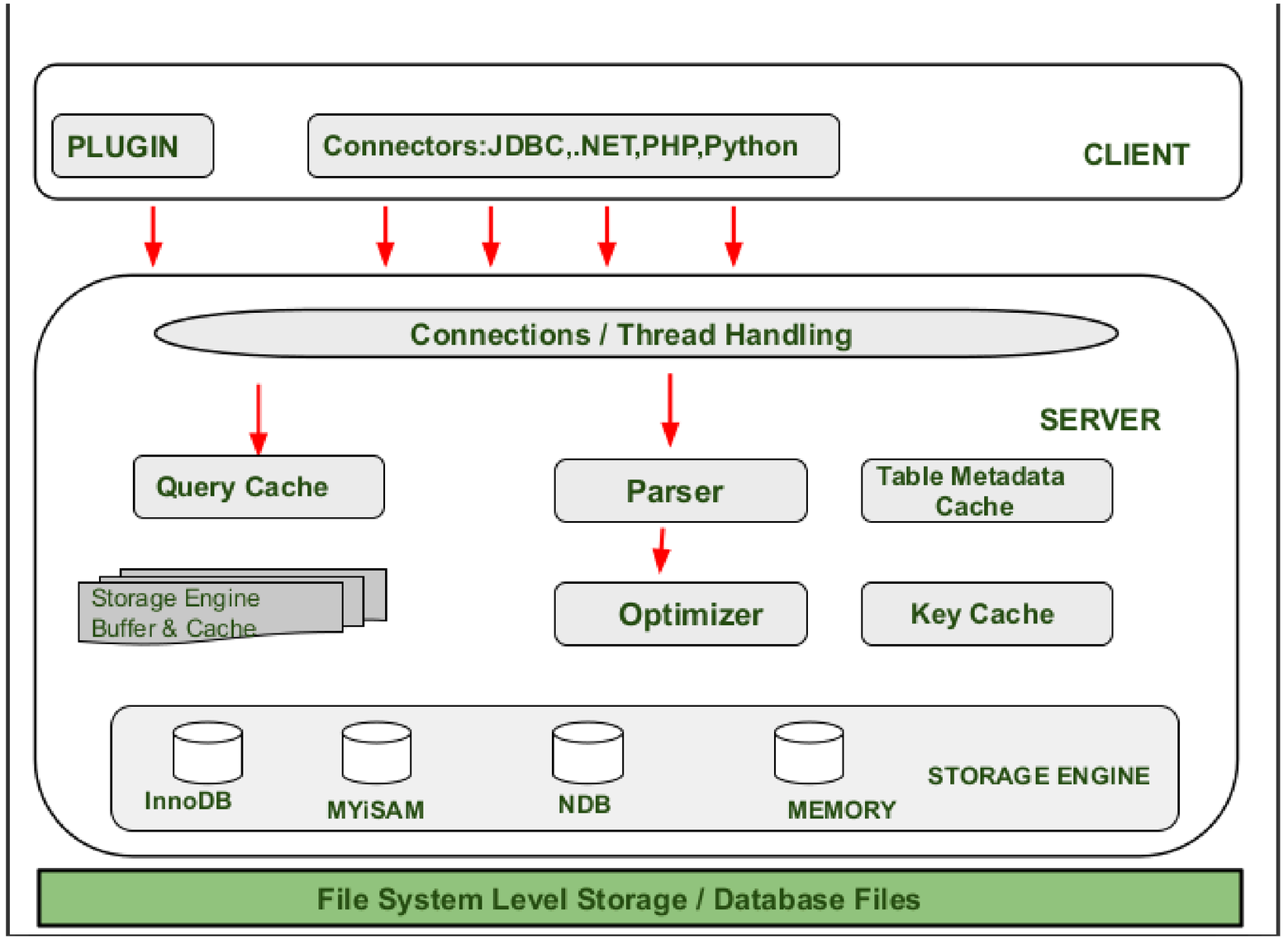
MySQL o Oracle MySQL è un relational database management system (RDBMS) composto da un client a riga di comando e un server.

Ambo i costituenti sono multipiattaforma e sono disponibili ufficialmente su tutte le distribuzioni conosciute, quali Debian, Ubuntu e CentOS, sebbene lo abbiano sostanzialmente sostituito con MariaDB a partire dal 2012.

È software libero rilasciato a doppia licenza, compresa la GNU General Public License, sviluppato per essere il più possibile conforme agli standard ANSI SQL e ODBC SQL.

I sistemi e i linguaggi di programmazione che lo supportano sono molto numerosi, fra cui ODBC, Java, Mono, .NET, PHP, Python.

Architettura - MySQL



SQL

SQL è un linguaggio standard per l'archiviazione, la manipolazione e il recupero dei dati nei database.

Nonostante sia uno standard ANSI/ISO ci sono molte versioni SQL ed estensioni proprietarie dei vari software di database.

Un database si può riassumere in una serie di tabelle identificate con un nome e possono essere relazionate tra loro, queste tabelle sono divise in colonne per ogni attributo e in righe (record) per le informazioni che vogliamo salvare.

Code	Name	Continent	Region	SurfaceArea	IndepYear	Population	LifeExpectancy	GNP	GNPOld	LocalName
ABW	Aruba	North America	Caribbean	193.00	NULL	103000	78.4	828.00	793.00	Aruba
AFG	Afghanistan	Asia	Southern and Central Asia	652090.00	1919	22720000	45.9	5976.00	NULL	Afganistan/Afqanestan
AGO	Angola	Africa	Central Africa	1246700.00	1975	12878000	38.3	6648.00	7984.00	Angola
AIA	Anguilla	North America	Caribbean	96.00	NULL	8000	76.1	63.20	NULL	Anguilla
ALB	Albania	Europe	Southern Europe	28748.00	1912	3401200	71.6	3205.00	2500.00	Shqipëria
AND	Andorra	Europe	Southern Europe	468.00	1278	78000	83.5	1630.00	NULL	Andorra
ANT	Netherlands Antilles	North America	Caribbean	800.00	NULL	217000	74.7	1941.00	NULL	Nederlandse Antillen

LE QUERY

L'interrogazione di un database avviene mediante una query.

Ovvero una espressione o una serie di espressioni con delle richieste da fare al database per avere una risposta, sotto forma di tabella.

```
1 •    SELECT * FROM world.country;
```

Questa query sta interrogando il database "world" andando a selezionare la tabella "country" dando quindi come risultato direttamente le righe della tabella country.

Code	Name	Continent	Region	SurfaceArea	IndepYear	Population	LifeExpectancy	GNP	GNPOld	LocalName
ABW	Aruba	North America	Caribbean	193.00	NULL	103000	78.4	828.00	793.00	Aruba
AFG	Afghanistan	Asia	Southern and Central Asia	652090.00	1919	22720000	45.9	5976.00	NULL	Afganistan/Afqanestan
AGO	Angola	Africa	Central Africa	1246700.00	1975	12878000	38.3	6648.00	7984.00	Angola
AIA	Anguilla	North America	Caribbean	96.00	NULL	8000	76.1	63.20	NULL	Anguilla
ALB	Albania	Europe	Southern Europe	28748.00	1912	3401200	71.6	3205.00	2500.00	Shqipëria
AND	Andorra	Europe	Southern Europe	468.00	1278	78000	83.5	1630.00	NULL	Andorra
ANT	Netherlands Antilles	North America	Caribbean	800.00	NULL	217000	74.7	1941.00	NULL	Nederlandse Antillen

ESEMPIO DI QUERY

In questo esempio andiamo a recuperare solo la colonna "Continent" della tabella country dando come risultato una tabella con una singola colonna con solo i valori di Continent di tutte le righe di country.

Continent
North America
Asia
Africa
North America
Europe
Europe

```
1 •   SELECT Continent
2     FROM world.country;
```

Code	Name	Continent	Region
ABW	Aruba	North America	Caribbean
AFG	Afghanistan	Asia	Southern and Central Asia
AGO	Angola	Africa	Central Africa
AIA	Anguilla	North America	Caribbean
ALB	Albania	Europe	Southern Europe
AND	Andorra	Europe	Southern Europe

SQL: SINTASSI

SQL è un linguaggio da una sintassi molto specifica che non è sensibile alle variazioni tra MAIUSCOLE e minuscole: "select" e "SELECT" sono uguali. Non è lo stesso per il nome delle tabelle e delle colonne.

Alcuni sistemi di database richiedono un punto e virgola alla fine di ogni istruzione SQL.

Il punto e virgola è il modo standard per separare ogni istruzione SQL nei database che consentono l'esecuzione di più istruzioni SQL nella stessa chiamata al server (come MySQL).

SQL: SINTASSI

Alcuni dei comandi SQL più importanti:

- **SELECT - estrae i dati da un database**
- **UPDATE - aggiorna i dati in un database**
- **DELETE - elimina i dati da un database**
- **INSERT INTO - inserisce nuovi dati in un database**
- **CREATE DATABASE - crea un nuovo database**
- **ALTER DATABASE - modifica un database**
- **CREATE TABLE - crea una nuova tabella**
- **ALTER TABLE - modifica una tabella**
- **DROP TABLE - elimina una tabella**
- **CREATE INDEX - crea un indice (chiave di ricerca)**
- **DROP INDEX - elimina un indice**

SQL: COMMENTI

Come in Java è possibile inserire del testo che viene ignorato.
In SQL si può commentare una riga inserendo "--" prima del testo da commentare:

```
1. -- selezioniamo tutte le colonne
2. SELECT *  FROM table_name -- dalla tabella
```

È possibile anche fare un commento multi riga inserendolo tra "/*" e "*/".

```
1. /* selezioniamo tutte le colonne
2. SELECT *  FROM table_name */
3.  SELECT *  FROM world.country -- dalla tabella country del database world
```

SQL: SELECT

L'istruzione SELECT serve per selezionare i dati da un database.

```
1. SELECT * FROM table_name;
```

I dati restituiti vengono archiviati in una tabella temporanea, denominata result-set.

```
1. SELECT column1, column2, ...
2. FROM table_name;
```

Con SELECT andiamo a selezionare le colonne di una tabella andando a specificare il nome, invece il "*" serve ad indicare tutte le colonne.
Con la "," possiamo selezionare più colonne contemporaneamente.

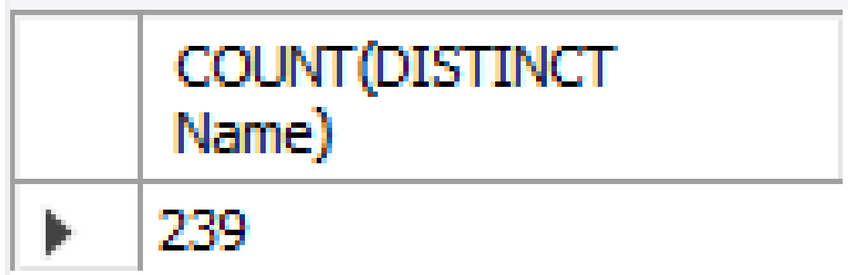
SQL: SELECT DISTINCT

L'istruzione SELECT DISTINCT funziona esattamente come SELECT solo che restituisce valori distinti tra loro, quindi senza ripetizioni.

```
1. SELECT DISTINCT column1, column2, ...
2. FROM table_name;
```

Con SELECT DISTINCT possiamo ad esempio sapere il numero di valori unici di una colonna usando il comando COUNT che conta i valori.

```
1. SELECT COUNT(DISTINCT Name) FROM world.country;
```



	COUNT(DISTINCT Name)
▶	239

SQL: WHERE Clause

L'istruzione WHERE è definita una "clausula", serve per mettere una condizione in modo da filtrare i record che andranno nel result-set.

```
1. SELECT column1, column2, ...
2. FROM table_name
3. WHERE condition;
```

Le condizioni possono essere di vario tipo e vanno scritte insieme all'operatore WHERE, ad esempio con valori stringa che vanno scritti tra singoli apici come ('testo').

```
1. SELECT *  FROM world.country
2. WHERE Region ='Antarctica';
```

SQL: WHERE Clause

Un altro esempio può essere con un valore numerico:

```
1. SELECT *  FROM world.country
2. WHERE Population=0;
```

Le operazioni che si possono fare come condizione sono:

- **=** - uguale;
- **>** - maggiore di;
- **<** - minore di;
- **>=** - maggiore uguale di;
- **<=** - minore uguale di;
- **<>** - non uguale(in alcune versione di SQL si scrive "**!=**", in altre ancora accetta entrambe le scritture come in MySQL);

- **BETWEEN** - tra un certo intervallo ("**BETWEEN** 0 **and** 10", valori compresi);
- **LIKE** - cerca un pattern, ovvero cerca una eguaglianza parziale o totale specificando il pattern fisso da trovare e le parti del dato variabili con un *metacarattere* come il "%" (esempio: "City LIKE 's%' " per trovare i valori che iniziano con "s" o "S");
- **IN** - per specificare più valori possibili (esempio "**IN** (0, 1000)");

SQL: WHERE (IL VALORE NULL)

Un campo con valore NULL è un campo senza valore che è diverso da un valore zero o da un campo che contiene spazi.

Un campo con un valore NULL è un campo che è stato lasciato vuoto durante la creazione di record.

Non è possibile verificare i valori NULL con operatori di confronto (come =, < o <>). Bisogna usare gli operatori IS NULL & IS NOT NULL.

```
1. SELECT column_names
2. FROM table_name
3. WHERE column_name IS NULL;
```

SQL: AND - OR - NOT

Questi operatori permettono di combinare più condizioni, stanno ad indicare:

AND - visualizza il record se tutte le condizioni da sono soddisfatte;

OR - visualizza il record se almeno una condizione è soddisfatta;

NOT - visualizza il record se la condizione NON sono soddisfatte;

```
1. SELECT column1, column2, ...
2. FROM table_name
3. WHERE condition1 AND condition2 AND condition3 ...;
```

Questi operatori possono essere affiancati da parentesi per indicare un gruppo di condizioni

```
1. SELECT * FROM table_name
2. WHERE condition1 AND (condition2 OR NOT condition3);
```

SQL: ORDER BY

L'istruzione ORDER BY serve ad ordinare il result-set su una o più colonne con due possibili criteri:

- **ASC - crescente**
- **DESC - decrescente**

Non è obbligatorio specificare il criterio di ordinamento, di default prende il criterio crescente (ASC).

```
1. SELECT column1, column2, ...
2. FROM table_name
3. ORDER BY column1, column2, ... ASC|DESC;
```

SQL: ORDER BY

In questo esempio ordiniamo In base alla colonna Region (string) e con ulteriore ordinamento decrescente in base alla colonna SurfaceArea (int).

Così facendo avremo un result-set ordinato in base alla regione, e le regioni uguali ordinate in base alla superfice più ampia.

```
1. SELECT *  FROM world.country
2. ORDER BY Region, SurfaceArea DESC;
```

Region	SurfaceArea
Antarctica	13120000.00
Antarctica	7780.00
Antarctica	3903.00
Antarctica	359.00
Antarctica	59.00
Australia and New Zealand	7741220.00
Australia and New Zealand	270534.00
Australia and New Zealand	135.00
Australia and New Zealand	36.00
Australia and New Zealand	14.00
Baltic Countries	65301.00
Baltic Countries	64589.00

SQL: GROUP BY

L'istruzione GROUP BY serve ad raggruppare i record con stessi valori del result-set su una o più colonne.

Viene spesso utilizzato con funzioni per raggruppare il result-set.

```
1. SELECT column_name(s)
2. FROM table_name
3. WHERE condition
4. GROUP BY column_name(s)
5. ORDER BY column_name(s);
```

SQL: GROUP BY

Un esempio:

```
1. SELECT Country, COUNT(CustomerID)
2. FROM Customers
3. GROUP BY Country;
```

Country	COUNT(CustomerID)
Argentina	3
Austria	2
Belgium	2
Brazil	9
Canada	3

SQL: GROUP BY

Un esempio con l'ORDER BY:

```
1. SELECT Country, COUNT(CustomerID)
2. FROM Customers
3. GROUP BY Country
4. ORDER BY COUNT(CustomerID) DESC;
```

Country	COUNT(CustomerID)
USA	13
Germany	11
France	11
Brazil	9
UK	7

Esercizi:

1) Utilizzo di DISTINCT e WHERE
Elencare, senza ripetizioni, tutte le regioni (Region) dei paesi che appartengono al continente (Continent) 'Europe'.

2) Combinazione di WHERE, ORDER BY
Elencare i nomi (Name) e la popolazione (Population) delle città (City) degli Stati Uniti (CountryCode = 'USA') che hanno una popolazione superiore a 1.000.000 abitanti, ordinando i risultati dalla città più popolosa alla meno popolosa.

3) GROUP BY con funzioni di aggregazione
Mostrare per ogni continente (Continent) presente nella tabella Country:

- **Il numero totale di paesi appartenenti a ciascun continente.**
- **La popolazione totale del continente.**

Ordinare il risultato per popolazione totale in ordine decrescente.

Esercizi:

4. **Combinazione di WHERE, GROUP BY e ORDER BY**

Mostrare, per ogni CountryCode, il numero di città presenti nella tabella City che hanno una popolazione superiore a 500.000 abitanti.
Visualizzare:

- **Il codice del paese (CountryCode);**
- **Il numero di città trovate;**
- **La popolazione totale delle città selezionate.**

Ordinare il risultato dal paese con il maggior numero di città a quello con il minor numero.

SQL: INSERT INTO

L'istruzione INSERT INTO serve per inserire nuovi record in una tabella.

```
1. INSERT INTO table_name (column1, column2, column3, ...)
2. VALUES (value1, value2, value3, ...);
```

È possibile non specificare le colonne se aggiungiamo valori a tutte le colonne ma bisogna assicurarsi che i valori abbiano l'ordine corretto delle colonne in cui devono essere inseriti.

```
1. INSERT INTO table_name
2. VALUES (value1, value2, value3, ...);
```

SQL: UPDATE

L'istruzione UPDATE serve per modificare i record esistenti in una tabella. Viene tendenzialmente usato insieme a WHERE per filtrare i record da modificare altrimenti modificherebbe tutti i record della tabella.

```
1. UPDATE table_name
2. SET column1 = value1, column2 = value2, ...
3. WHERE condition;
```

Con SET andiamo a specificare i valori da cambiare indicando la colonna con l'assegnazione del valore.

```
1. UPDATE Customers
2. SET ContactName = 'Alfred Schmidt', City= 'Frankfurt'
3. WHERE CustomerID = 1;
```

SQL: DELETE

L'istruzione DELETE serve per eliminare i record esistenti in una tabella.
Viene tendenzialmente usato insieme a WHERE per filtrare i record da eliminare altrimenti eliminerebbe tutti i record della tabella.

```
1. DELETE FROM table_name
2. WHERE condition;
```

Questa istruzione va ad eliminare solo i record, la struttura della tabella rimane invariata pure se elimiano tutti i record senza specificare una condizione.

```
1. DELETE FROM Customers
2. WHERE CustomerName='Mario Rossi';
```

Esercizi:

Esercizio sui comandi SQL: GROUP BY, ORDER BY e INSERT INTO

Si consideri un database che contiene informazioni su una libreria. Nel database è presente una tabella chiamata Libri con la seguente struttura:

```
Libri (
    id INT PRIMARY KEY,
    titolo VARCHAR(100),
    autore VARCHAR(100),
    genere VARCHAR(50),
    prezzo DECIMAL(5,2),
    anno_pubblicazione INT
)
```

1) Inserimento dati (INSERT INTO)
Inserire almeno 6 nuovi libri nella tabella Libri usando il comando SQL INSERT INTO.
I libri devono appartenere a generi e autori diversi, ed essere pubblicati in anni differenti.

2) Aggregazione e raggruppamento (GROUP BY)
Scrivere una query che, usando il comando GROUP BY, mostri per ogni genere:

- **il numero totale di libri presenti;**
- **il prezzo medio dei libri appartenenti a quel genere.**

La query dovrà restituire il risultato ordinato alfabeticamente per genere.

3) Ordinamento risultati (ORDER BY)
Scrivere una query che elenchi tutti i libri pubblicati dopo l'anno 2010 ordinati in modo decrescente per anno di pubblicazione e, in caso di anno uguale, in ordine crescente per prezzo.

SQL: SELECT TOP (LIMIT)

L'istruzione SELECT TOP serve per specificare il numero di record da restituire.

Non tutti i sistemi di database supportano questa istruzione. MySQL supporta l'operazione LIMIT per selezionare un numero limitato di record.

```
1. SELECT column_name
2. FROM table_name
3. WHERE condition
4. LIMIT number;
```

LIMIT quindi ci permette di indicare il numero massimo di record da recuperare da un database, questa operazione tendenzialmente si esegue per tabelle con un grande numero di record ottimizzando le prestazioni.

```
1. SELECT * FROM Customers
2. LIMIT 50;
```

SQL: ALIAS

Gli ALIAS SQL servono per assegnare un nome temporaneo ad un elemento come una tabella, una colonna, il risultato di una funzione ecc ecc...; In questo modo possiamo richiamare direttamente il nome temporaneo assegnato all'elemento.

Gli ALIAS possono essere utili quando:

- **sono coinvolte più tabelle in una QUERY;**
- **vengono usate le funzioni in una QUERY;**
- **i nomi delle colonne sono grandi o poco leggibili;**
- **due o più colonne vengono combinate insieme;**

Per assegnare un ALIAS viene usata la parola chiave AS quando andiamo a selezione o richiamare in SELECT o in FROM l'elemento a cui assegnare il nome:

```
1. SELECT column_name AS alias_columName
2. FROM table_name AS alias_tableName;
```

SQL: MIN() - MAX()

Sono delle funzioni che restituiscono rispettivamente il valore minimo e il valore massimo di una colonna, hanno lo stesso tipo di sintassi:

```
1. SELECT MIN(column_name)
2. FROM table_name
3. WHERE condition;
```

Nel prossimo esempio prenderemo il prezzo più alto da una tabella di prodotti e assoceremo il risultato ad un nome temporaneo tramite la parola chiave AS (ovvero un ALIAS):

```
1. SELECT MAX(Price) AS LargestPrice
2. FROM Products;
```

SQL: COUNT() - AVG() - SUM()

Sono delle funzioni che eseguono un calcolo restituendo un valore specifico:

- **COUNT() - restituisce il numero di record selezionati;**
- **AVG() - restituisce il valore medio di una colonna con valori numerici;**
- **SUM() - restituisce la somma totale di una colonna con valori numerici;**

Tutte e tre le funzioni hanno lo stesso tipo di sintassi e possono essere associate a delle condizioni con WHERE:

```
1. SELECT SUM(column_name)
2. FROM table_name
3. WHERE condition;
```

Esercizio

Si consideri una tabella chiamata Vendite con la seguente struttura, almeno 20 elementi generati:

```
Vendite (
    id INT,
    prodotto VARCHAR(100),
    categoria VARCHAR(50),
    quantita INT,
    prezzo_unitario DECIMAL(6,2),
    data_vendita DATE
)
```

Scrivi le query SQL per rispondere alle seguenti richieste:

- **Totale vendite per categoria**

Visualizza, per ogni categoria, il numero totale di vendite effettuate.

- **Prezzo medio per categoria**

Mostra, per ogni categoria, il prezzo medio dei prodotti venduti.

- **Quantità totale venduta per ogni prodotto**

Mostra il totale delle quantità vendute (SUM) per ciascun prodotto.

- **Prezzo massimo e minimo venduto nella tabella**

Mostra il prezzo massimo e il prezzo minimo tra tutti i prodotti venduti.

- **Numero totale di righe nella tabella**

Conta quante vendite sono state registrate nella tabella Vendite.

- **I 5 prodotti più costosi (in base al prezzo_unitario)**

Elenca i 5 prodotti più costosi ordinati in modo decrescente rispetto al prezzo.

- **I 3 prodotti meno venduti per quantità totale**

Mostra i nomi dei 3 prodotti con la quantità totale più bassa venduta (usa SUM e LIMIT).

SQL: WILDCARD CHARACTERS

In italiano metacaratteri, sono dei caratteri che servono per sostituire uno o più caratteri in una stringa.

Vengono utilizzati nelle condizioni come WHERE e LIKE sono:

- **"%" - zero o più caratteri ("bl%" = bl, black, blue e blob);**
- **"_" - un singolo carattere ("h_t" = hot, hat e hit);**

Il resto dei metacaratteri hanno bisogno della funzione "REGEXP_LIKE(colonna, pattern)"

- **"[]" - ogni singolo carattere all'interno delle parentesi ("h[oa]t" = hot e hat);**
- **"^" - qualsiasi carattere non compreso tra parentesi ("h[^oa]t" = hit);**
- **"-" - qualsiasi carattere all'interno del range specificato ("c[a-b]t" = cat e cbt);**

I metacaratteri possono essere usati in combinazione come ad esempio '_r%' che rappresenta valori con almeno una r al secondo carattere.

SQL: LIKE

È un operatore che cerca un pattern preciso usando anche i metacaratteri:

```
1. SELECT column1, column2, ...
2. FROM table_name
3. WHERE columnN LIKE pattern;
```

In questo esempio seleziona tutti i clienti con il nome che inizia per "a":

```
1. SELECT * FROM Customers
2. WHERE CustomerName LIKE 'a%';
```

SQL: IN

È un operatore che consente di specificare più valori in una clausola, una scorciatoia per più condizioni OR.

```
1. SELECT column_name(s)
2. FROM table_name
3. WHERE column_name IN (value1, value2, ...);
```

È possibile inserire anche una table all'interno dei valori possibili:

```
1. SELECT column_name(s)
2. FROM table_name
3. WHERE column_name IN (SELECT STATEMENT);
```

SQL: IN

Esempi di utilizzo:

```
1. SELECT * FROM Customers
2. WHERE Country IN ('Germany', 'France', 'UK');
```

con NOT:

```
1. SELECT * FROM Customers
2. WHERE Country NOT IN ('Germany', 'France', 'UK');
```

Confrontando valori di una table con un'altra table:

```
1. SELECT * FROM Customers
2. WHERE Country IN (SELECT Country FROM Suppliers);
```

SQL: BETWEEN

È un operatore che seleziona i valori (come: numeri, testo o date) all'interno di un determinato intervallo. L'operatore è inclusivo ovvero sono inclusi i valori di inizio e fine.

```
1. SELECT column_name(s)
2. FROM table_name
3. WHERE column_name BETWEEN value1 AND value2;
```

Un esempio di utilizzo con più condizioni:

```
1. SELECT * FROM Products
2. WHERE Price BETWEEN 10 AND 20
3. AND CategoryID NOT IN (1,2,3);
```

SQL: BETWEEN

Un esempio utilizzando le date:

```
SELECT * FROM Orders
WHERE OrderDate BETWEEN #07/01/1996# AND #07/31/1996#;
```

Un'altra sintassi per le date è:

```
SELECT * FROM Orders
WHERE OrderDate BETWEEN '1996-07-01' AND '1996-07-31';
```

Un esempio utilizzando un ORDER BY:

```
SELECT * FROM Products
WHERE ProductName NOT BETWEEN 'Carnarvon Tigers' AND 'Mozzarella di Giovanni'
ORDER BY ProductName;
```

Esercizio

Si consideri una tabella chiamata Clienti con la seguente struttura, almeno 20 dati inseriti:

```
Clienti (
    id INT,
    nome VARCHAR(100),
    cognome VARCHAR(100),
    email VARCHAR(100),
    eta INT,
    citta VARCHAR(100)
)
```

Scrivere le query SQL per rispondere alle seguenti richieste:

- **Clienti con email su dominio Gmail**
  Seleziona tutti i clienti la cui email termina con @gmail.com.
- **Clienti con nome che inizia con la lettera 'A'**
  Mostra tutti i clienti il cui nome comincia con la lettera A.
- **Clienti con cognome che contiene esattamente 5 lettere**
  Mostra tutti i clienti il cui cognome è composto da esattamente 5 caratteri.
- **Clienti con età compresa tra 30 e 40 anni (inclusi)**
  Elenca i clienti che hanno un'età compresa tra 30 e 40 anni, inclusi gli estremi.
- **Clienti che vivono in città il cui nome contiene 'roma' (maiuscole/minuscole ignorate)**
  Mostra tutti i clienti che abitano in una città il cui nome contiene la stringa roma, indipendentemente da maiuscole o minuscole.

SQL: JOIN

È una clausula che viene utilizzata per combinare righe da due o più tabelle, in base a una colonna correlata tra di loro.

```
1. SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
2. FROM Orders
3. INNER JOIN Customers ON Orders.CustomerID=Customers.CustomerID;
```

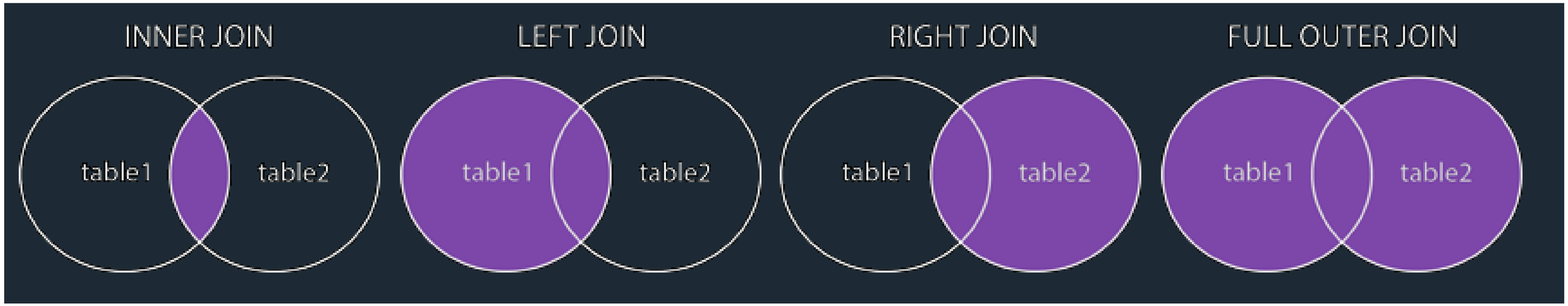
In questo esempio stiamo andando a prendere i record che hanno lo stesso valore CustomerID unendo i valori selezionati generando un result-set con 3 colonne:

1. **OrderID della tabella orders**
2. **CustomerName della tabella customers**
3. **OrderDate della tabella orders**

SQL: JOIN

Ci sono 4 tipi di join:

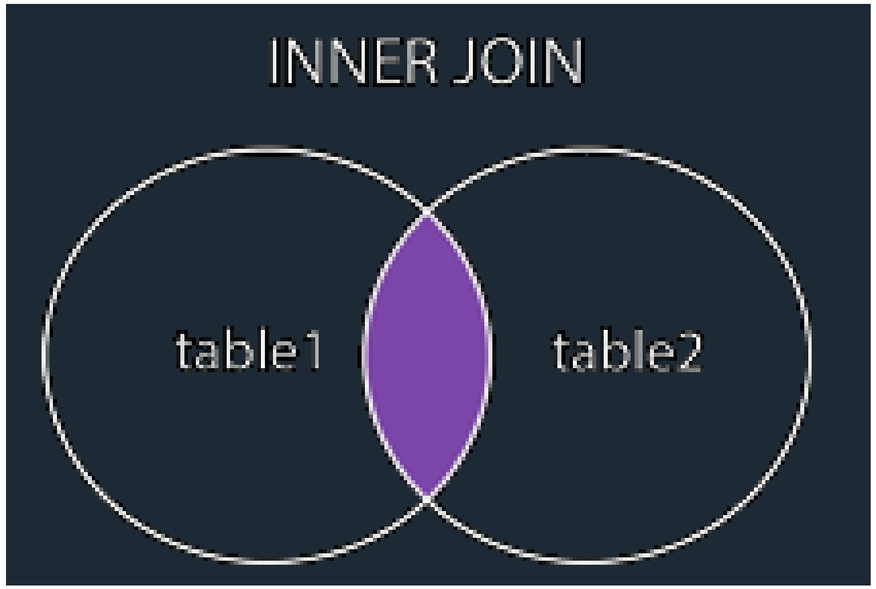
- **(INNER) JOIN: restituisce record con valori in comune**
- **LEFT (OUTER) JOIN: restituisce tutti i record della tabella di sinistra e i record con valori in comune**
- **RIGHT (OUTER) JOIN: restituisce tutti i record della tabella di destra e i record con valori in comune**
- **FULL (OUTER) JOIN: restituisce tutti i record quando è presente una corrispondenza in uno dei due a sinistra o nella tabella a destra**



SQL: INNER JOIN

INNER JOIN seleziona i record con valori corrispondenti in entrambe le tabelle.

```
1. SELECT column_name(s)
2. FROM table1
3. INNER JOIN table2
4.    ON table1.column_name = table2.column_name;
```



SQL: INNER JOIN

Esempio di utilizzo:

```
1. SELECT Orders.OrderID, Customers.CustomerName
2. FROM Orders
3. INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID;
```

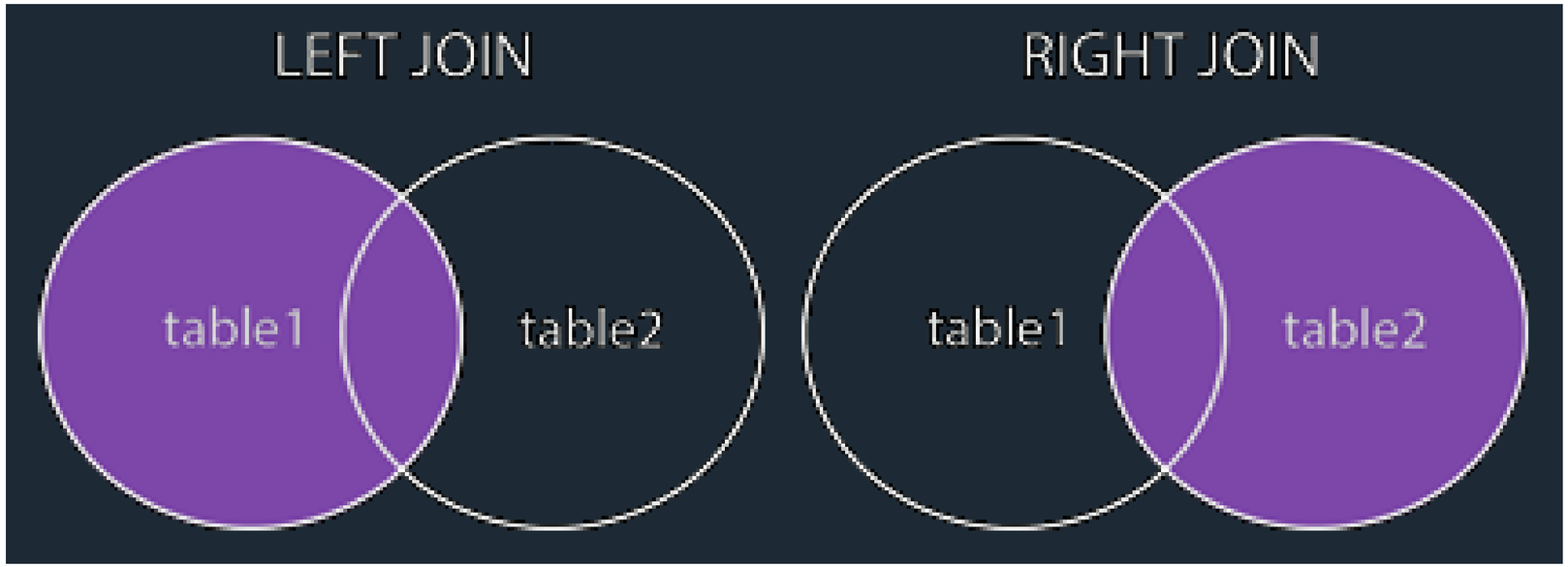
Esempio di utilizzo con più join (da notare le parentesi):

```
1. SELECT Orders.OrderID, Customers.CustomerName, Shippers.ShipperName
2. FROM ((Orders
3. INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID)
4. INNER JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID);
```

SQL: LEFT JOIN - RIGHT JOIN

Questi join restituiscono i record della tabella di riferimento (Left o Right in base al tipo di join) insieme ai record che con i valori corrispondenti.

```
1. SELECT column_name(s)
2. FROM table1
3. LEFT JOIN table2
4. ON table1.column_name = table2.column_name;
```



SQL: LEFT JOIN - RIGHT JOIN

Un esempio dove recuperiamo e ordiniamo tutti i clienti in aggiunta recuperiamo anche l'eventuale id dell'ordine:

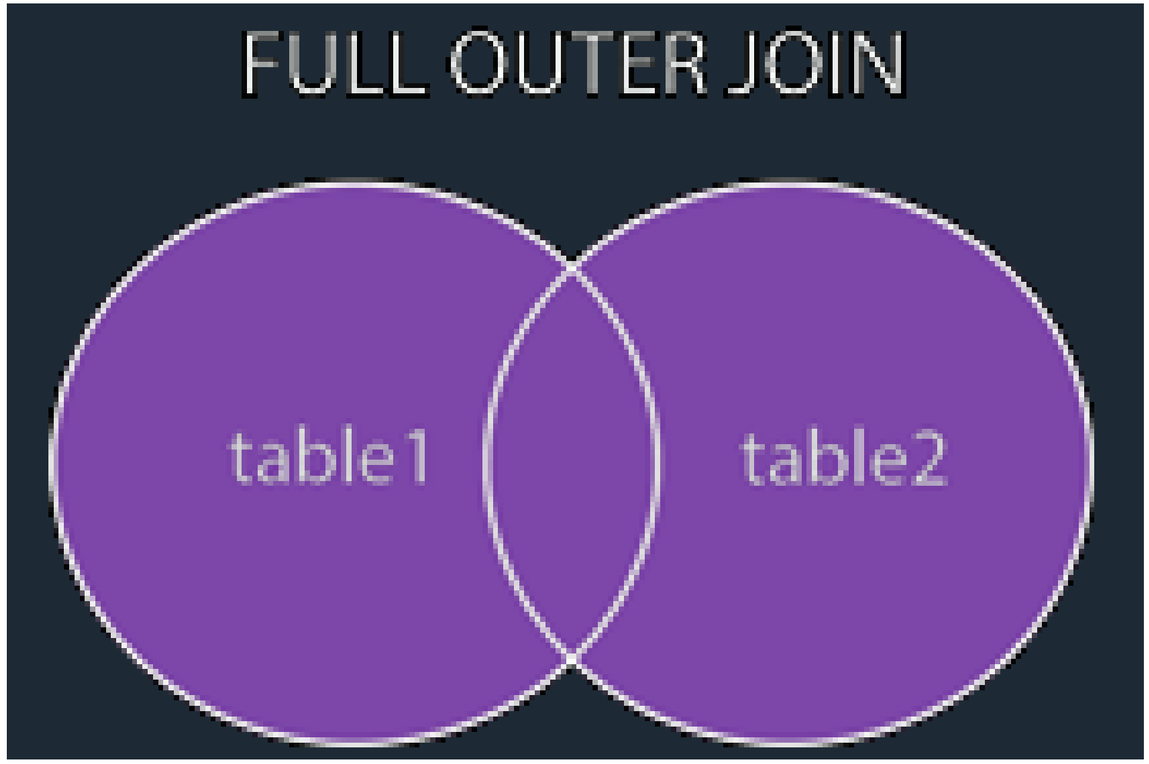
```
1. SELECT Orders.OrderID,  Customers.CustomerName
2. FROM Customers
3. LEFT JOIN Orders
4. ON Customers.CustomerID=Orders.CustomerID
5. ORDER BY Customers.CustomerName;
```

OrderID	CustomerName
null	Alfreds Futterkiste
10308	Ana Trujillo Emparedados y helados
10365	Antonio Moreno Taquería

SQL: CROSS JOIN

CROSS JOIN restituisce tutti i record da entrambe le tabelle.

```
1. SELECT column_name(s)
2. FROM table1
3. CROSS JOIN table2;
```



Esercizio

Si considerino le seguenti due tabelle con 20 dati l'una:

```
1. Clienti (
2.     id INT,
3.     nome VARCHAR(100),
4.     città VARCHAR(100)
5. )
```

```
1. Ordini (
2.     id INT,
3.     id_cliente INT,
4.     data_ordine DATE,
5.     importo DECIMAL(7,2)
6. )
```

Le due tabelle sono collegate dalla relazione tra Clienti.id e Ordini.id_cliente.

Esercizio 1 – INNER JOIN Obiettivo:
Visualizza l'elenco dei clienti che hanno effettuato almeno un ordine.
Per ciascuno, mostra: nome del cliente, data dell'ordine e importo.

Esercizio 2 – LEFT JOIN Obiettivo:
Visualizza tutti i clienti, inclusi quelli che non hanno mai effettuato ordini.
Mostra per ciascuno: nome del cliente, data dell'ordine (se presente) e importo (se presente).

Esercizio 3 – RIGHT JOIN Obiettivo:
Visualizza tutti gli ordini, anche quelli che non hanno un cliente associato (caso anomalo).
Mostra per ciascuno: nome del cliente (se esiste), data dell'ordine e importo.

Esercizio

Devi realizzare un report completo per il reparto vendite, che soddisfi tutte le seguenti condizioni usando correttamente e separatamente i tre tipi di JOIN:

- **Elenca i clienti attivi, cioè quelli che hanno effettuato almeno un ordine, mostrando per ciascuno:**
  - **Nome del cliente**
  - **Totale ordini effettuati**
  - **Somma totale degli importi spesi**

- **Elenca i clienti inattivi, cioè quelli che non hanno mai effettuato ordini, mostrando solo:**
  - **Nome del cliente**
  - **Città di residenza**

- **Individua gli ordini orfani, cioè ordini presenti in tabella ma senza un cliente valido associato (es. cliente cancellato), e mostra:**
  - **ID dell'ordine**
  - **Data dell'ordine**
  - **Importo**
  - **(Cliente = NULL)**

Requisiti tecnici:

- **Per il punto 1: usa INNER JOIN.**
- **Per il punto 2: usa LEFT JOIN con condizione su IS NULL.**
- **Per il punto 3: usa RIGHT JOIN con condizione su IS NULL.**

Esercizio TEST:

Esercizio 1 – INNER JOIN + WHERE + LIKE
Visualizzare l’elenco dei libri venduti in almeno un negozio, mostrando:
titolo del libro, autore, data_vendita, negozio.
Includere solo i libri il cui autore contiene la stringa “King” (indipendentemente da maiuscole/minuscole).

Esercizio 2 – LEFT JOIN + WHERE + BETWEEN
Visualizzare tutti i libri, anche quelli che non hanno ancora vendite registrate, mostrando per ciascuno:
titolo, anno_pubblicazione, prezzo, data_vendita (se presente).
Filtrare i risultati per anno_pubblicazione compreso tra 2000 e 2010.

Esercizio 3 – INNER JOIN + WHERE + IN
Visualizzare i dati dei libri venduti nei negozi appartenenti a una lista specifica:
('Libreria Centrale', 'BookCity Milano', 'Cartoleria Roma').
Mostrare titolo, negozio, quantita, prezzo totale (quantita * prezzo).

Esercizio 4 – RIGHT JOIN + WHERE + LIKE + BETWEEN
Mostrare tutti i record di vendita, anche quelli che fanno riferimento a libri non più presenti nella tabella Libri (caso anomalo).
Mostrare: titolo (se esiste), data_vendita, prezzo, quantita.
Includere solo le vendite:
avvenute tra il 2020-01-01 e il 2022-12-31
presso negozi il cui nome contiene la parola “Book” (case-insensitive).

Esercizio 5 – INNER JOIN + WHERE combinato
Mostrare titolo, autore, prezzo e data_vendita dei libri:
con genere IN (‘Fantasy’, ‘Horror’, ‘Giallo’)
pubblicati dopo il 2015,
venduti in negozi il cui nome contiene ‘Store’,
ordinati dal più recente al più vecchio.

Si considerino le seguenti due tabelle, ciascuna con almeno 30 dati inseriti:

```
Libri (
  id INT,
  titolo VARCHAR(100),
  autore VARCHAR(100),
  genere VARCHAR(50),
  anno_pubblicazione INT,
  prezzo DECIMAL(6,2)
)

Vendite (
  id INT,
  id_libro INT,
  data_vendita DATE,
  quantita INT,
  negozio VARCHAR(100)
)
```

SQL: UNION

L'operatore UNION viene utilizzato per combinare il result-set di risultati distinti di due o più istruzioni. Vanno però rispettate queste condizioni:

- **L’istruzione SELECT all'interno deve avere lo stesso numero di colonne;**
- **Le colonne devono inoltre avere tipi di dati simili;**
- **Le colonne devono inoltre essere nello stesso ordine.**

Sintassi:

```
1. SELECT column_name(s) FROM table1
2. UNION
3. SELECT column_name(s) FROM table2;
```

SQL: UNION ALL

È identico all'operatore UNION solo che include tutti i valori anche i duplicati.

```
1. SELECT column_name(s) FROM table1
2. UNION ALL
3. SELECT column_name(s) FROM table2;
```

Esempio:

```
• SELECT City FROM Customers
• UNION ALL
• SELECT City FROM Suppliers
• ORDER BY City;
```

SQL: HAVING

HAVING è una clausola che opera sui raggruppamenti invece che sui campi della tabella.

```
1. SELECT column_name(s)
2. FROM table_name
3. WHERE condition
4. GROUP BY column_name(s)
5. HAVING condition
6. ORDER BY column_name(s);
```

NOTA: deve essere usato prima dell'ORDER By.

SQL: EXISTS

L'operatore EXISTS viene utilizzato per verificare l'esistenza di qualsiasi record in una sottoquery, restituisce TRUE se la sottoquery restituisce uno o più record.

```
1. SELECT column_name(s)
2. FROM table_name
3. WHERE EXISTS
4. (SELECT column_name FROM table_name WHERE condition);
```

L'esempio seguente restituisce TRUE ed elenca i fornitori con un prezzo del prodotto inferiore a 20:

```
1. SELECT SupplierName
2. FROM Suppliers
3. WHERE EXISTS
4. (SELECT ProductName FROM Products WHERE Products.SupplierID =
5. Suppliers.supplierID AND Price < 20);
```

SQL: ANY

L'operatore ANY restituisce TRUE se UNO qualsiasi dei valori della sottoquery soddisfa la condizione dell'operatore.

```
1. SELECT column_name(s)
2. FROM table_name
3. WHERE column_name operator ANY
4. (SELECT column_name FROM table_name WHERE condition);
```

L'esempio seguente restituisce TRUE se qualsiasi prodotto ha ordini superiori a 90:

```
1. SELECT ProductName
2. FROM Products
3. WHERE ProductID = ANY
4. (SELECT ProductID FROM OrderDetails WHERE Quantity > 90);
```

SQL: ALL

L'operatore ALL restituisce TRUE se TUTTI i valori della sottoquery soddisfa la condizione dell'operatore, si può utilizzare in: SELECT, WHERE, HAVING.

```
1. SELECT ALL column_name(s)
2. FROM table_name
3. WHERE column_name operator ALL
4. (SELECT column_name FROM table_name WHERE condition);
```

L'esempio seguente elenca ProductName se TUTTI i record nella tabella OrderDetails ha Quantity uguale a 10:

```
1. SELECT ProductName
2. FROM Products
3. WHERE ProductID = ALL (SELECT ProductID FROM OrderDetails WHERE Quantity = 10);
```

Esercizio
Si considerino le seguenti due tabelle con 20 dati l'una:
1. Clienti (
2. id INT,
3. nome VARCHAR(100),
4. città VARCHAR(100)
5.)

1. Ordini (
2. id INT,
3. id_cliente INT,
4. data_ordine DATE,
5. importo DECIMAL(7,2)
6.)

Le due tabelle sono collegate dalla relazione tra Clienti.id e Ordini.id_cliente.
– UNION e UNION ALL :
Visualizza l'elenco delle città in cui sono presenti clienti unendolo a una lista di città target, prima rimuovendo i duplicati e successivamente mantenendoli tutti.
– HAVING :
Visualizza l'elenco dei clienti che hanno effettuato una spesa totale complessiva superiore a 1.000€.
Mostra per ciascuno: id_cliente e totale speso.
– EXISTS :
Visualizza tutti i clienti che hanno registrato almeno un ordine nel sistema.
Mostra per ciascuno: id e nome del cliente.
– ANY e ALL :
Visualizza gli ordini con importo superiore ad almeno un ordine (ANY) o a tutti gli ordini (ALL) effettuati dai clienti di una specifica città.
Mostra per ciascuno: id dell'ordine e importo.

NOTA: potete utilizzare USE e omettere il nome della tabella per le colonne quando possibile

Esercizi:

1) Si vogliono recuperare dal database "world" la lingua e la nazione di ogni città.

2) Si vuole recuperare il numero di città per nazione dal database "world" mostrando anche il nome della nazione e ordinarli in base al numero di città.

3) Si vuole conoscere la lista di repubbliche con aspettativa di vita maggiore dei 70 anni, inoltre si vuole visualizzare anche la lingua parlata.

ESERCIZIO 1:
Si vuole recuperare dal database WORLD le lingue parlate per nazione con la rispettiva percentuale di utilizzo;

ESERCIZIO 2:
Si vuole recuperare dal database WORLD le nazioni e la percentuale della lingua più parlata;

ESERCIZIO 3:
Unire gli esercizi 1e2 facendole diventare subQuery per mostrare la lingua più parlata di una nazione con la percentuale;

SQL: SCHEMA RELAZIONALE

È uno schema che ci riassume le varie tabelle con le PK (chiave primaria) e le FK* (chiave esterna):

- **ATTORI (CodAttore, Nome, AnnoNascita, Nazionalità);**
- **RECITA (CodAttore*, CodFilm*)**
- **FILM (CodFilm, Titolo, AnnoProduzione, Nazionalità, Regista, Genere)**
- **PROIEZIONI (CodProiezione, CodFilm*, CodSala*, Incasso, DataProiezione)**
- **SALE (CodSala, Posti, Nome, Città)**

SQL: INSERT INTO SELECT

INSERT INTO con SELECT copia i record da una tabella e li aggiunge ad un'altra tabella.

L'operazione richiede che I tipi di dati delle colonne nelle tabelle di origine e di destinazione corrispondono.

```
1. INSERT INTO table2 (column1, column2, column3, ...)
2. SELECT column1, column2, column3, ...
3. FROM table1
4. WHERE condition;
```

SQL: CASE

CASE passa attraverso dei WHEN restituisce con THEN un valore quando trova la prima condizione vera, quindi, una volta che una condizione è vera, si fermerà la lettura del CASE e restituirà un valore.

Se nessuna condizione è vera, restituisce il valore indicato nella clausola ELSE.

```
1. CASE
2.     WHEN condition1 THEN result1
3.     WHEN condition2 THEN result2
4.     WHEN conditionN THEN resultN
5.     ELSE result
6. END;
```

Possiamo quindi considerarlo un if-elseif-else oppure uno switch di java

SQL: CASE

Un esempio che passa un valore in base alla condizione vera trovata:

```
SELECT OrderID, Quantity,
CASE
    WHEN Quantity > 30 THEN 'The quantity is greater than 30'
    WHEN Quantity = 30 THEN 'The quantity is 30'
    ELSE 'The quantity is under 30'
END AS QuantityText
FROM OrderDetails;
```

SQL: CASE

Un esempio che sceglie in base alla condizione per quale colonna ordinare:

```
SELECT CustomerName, City, Country
FROM Customers
ORDER BY
(CASE
    WHEN City IS NULL THEN Country
    ELSE City
END);
```

SQL: IFNULL() - COALESCE()

IFNULL() è una funzione che restituisce un valore alternativo se l'espressione è NULL:

```
1. IFNULL(expression, alt_value)
```

COALESCE() è una funzione che restituisce il primo valore non NULL:

```
1. COALESCE(val1, val2, ...., val_n)
```

Esercizio1:
scrivere e mostrare con JDBC una query che vada a prendere le nazioni mostrando come riposta se hanno o meno una superfice di 100.000 e se hanno registrato un IndepYear, se è presente l'anno va mostrato altrimenti si indica che non è presente;

Esercizio2:
recuperare e mostrare le città con il codice della nazione che indica l'utente. Inoltre l'utente può decidere:

- **l'ordine con cui ordinare le città in maniera decrescente o meno;**
- **se filtrare le città per un minimo di popolazione;**
- **se mostrare il nome della nazione a cui fa riferimento il code;**

SQL: CREATE/DROP DATABASE

Sono delle operazioni che richiedono i privilegi di un amministratore.

CREATE DATABASE ci permette di creare un nuovo database:

```
1. CREATE DATABASE databasename;
```

DROP DATABASE ci permette di eliminare un database eliminando tutto il suo contenuto:

```
1. DROP DATABASE databasename;
```

SQL: CREATE TABLE

CREATE TABLE ci permette di creare una nuova tabella nel database database specificando il nome delle colonne con le loro caratteristiche:

```
1. CREATE TABLE table_name (
2.     column1 datatype vincoli,
3.     column2 datatype vincoli,
4.     column3 datatype vincoli,
5.    ....
6. );
```

Ci sono numerosi datatype che possiamo dare alle colonne, i più usati cono il VARCHAR(nMax di char) e l'INT.

SQL: CREATE TABLE

Esempio:

```
1. CREATE TABLE Persons (
2.     PersonID int,
3.     LastName varchar(255),
4.     FirstName varchar(255),
5.     Address varchar(255),
6.     City varchar(255)
7. );
```

PersonID	LastName	FirstName	Address	City

SQL: CREATE TABLE

CREATE TABLE ci permette di creare una nuova tabella anche da un'altra tabella, copiando le caratteristiche delle tabelle selezionate e i suoi record:

```
1. CREATE TABLE new_table_name
2.     SELECT column1, column2,...
3.     FROM existing_table_name
4.     WHERE ....;
```

Un esempio di sintassi:

```
1. CREATE TABLE TestTable
2. SELECT customername, contactname
3. FROM customers;
```

SQL: DATATYPE

In MySQL ci sono tre tipi di dati principali: stringa, numerico, data e ora. Qui elencheremo i principali.

I principali tipi sono:

- **CHAR(size): una stringa di grandezza fissa(max 255);**
- **VARCHAR(max_size): una stringa a grandezza variabile fino al massimo indicato(max 65535);**
- **MEDIUMTEXT: una stringa lunga fino a 16,777,215 caratteri;**
- **LONGTEXT: una stringa lungafino a 4,294,967,295 caratteri;**
- **ENUM(val1, val2, val3, ...): una lista di valori possibili;**
- **BOOL: un valore booleano, 0 per false e 1 per true;**
- **INT: un valore numerico intero;**
- **FLOAT: un valore numero con virgola mobile;**
- **DATE: data in formato YYYY-MM-DD;**

SQL: DROP/TRUNCATE TABLE

DROP TABLE ci permette di eliminare una tabella con tutto il suo contenuto:

```
1. DROP TABLE table_name
```

TRUNCATE TABLE ci permette di eliminare tutti i record di una tabella senza eliminare la tabella:

```
1. TRUNCATE TABLE table_name
```

SQL: ALTER TABLE

ALTER TABLE ci permette di aggiungere, eliminare o modificare colonne in una tabella esistente con:

- **ADD: per aggiungere una colonna;**
- **DROP COLUMN: per eliminare una colonna**
- **MODIFY COLUMN: per modificare una colonna;**

1. **ALTER TABLE table_name**
2. **ADD column_name datatype;**

NOTA: è possibile modificare anche i vincoli di una tabella.

SQL: CONSTRAINT

Per ogni colonna si possono specificare dei vincoli sul tipo di dato.

```
1. CREATE TABLE table_name (
2.     column1 datatype constraint,
3.     .... );
```

I vincoli più utilizzati sono:

- **NOT NULL - non ci può essere un valore NULL;**
- **UNIQUE - tutti i valori in una colonna sono differenti;**
- **PRIMARY KEY - identifica in modo univoco ogni riga di una tabella;**
- **FOREIGN KEY - identifica una chiave esterna;**
- **CHECK - i valori in una colonna devono soddisfare una condizione;**
- **DEFAULT - imposta un valore di default;**
- **CREATE INDEX - crea un index per recuperare dati più velocemente.**

SQL: CONSTRAINT

NOT NULL esempio:

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255) NOT NULL,
    Age int
);
```

Per poi modificare in successiva Age:

```
ALTER TABLE Persons
MODIFY Age int NOT NULL;
```

SQL: CONSTRAINT

UNIQUE esempio:

```
1. CREATE TABLE Persons (
2.     ID int NOT NULL,
3.     LastName varchar(255) NOT NULL,
4.     FirstName varchar(255),
5.     Age int,
6.     UNIQUE (ID) );
```

È possibile indicare più colonne UNIQUE modificando la sintassi indicando il nome del vincolo:

```
1. CONSTRAINT ctnt_name UNIQUE (ID,LastName)
```

Per poi eliminare un vincolo bisogna fare:

```
1. ALTER TABLE table_name
2. DROP INDEX ctnt_name;
```

SQL: CONSTRAINT

PRIMARY KEY esempio:

```
1. CREATE TABLE Persons (
2.     ID int NOT NULL,
3.     LastName varchar(255) NOT NULL,
4.     FirstName varchar(255),
5.     Age int,
6.     PRIMARY KEY (ID) );
```

È possibile indicare una PRIMARY KEY su più colonne con un CONSTRAINT:

```
1. CONSTRAINT PK_name PRIMARY KEY (column1, column2)
```

Per poi eliminare una PRIMARY KEY bisogna fare:

```
1. ALTER TABLE table_name
2. DROP PRIMARY KEY;
```

SQL: CONSTRAINT

FOREIGN KEY esempio:

```
1. CREATE TABLE Orders (
2.     OrderID int NOT NULL,
3.     OrderNumber int NOT NULL,
4.     PersonID int,
5.     PRIMARY KEY (OrderID),
6.     FOREIGN KEY (PersonID) REFERENCES Persons(PersonID) );
```

È possibile indicare una FOREIGN KEY con un CONSTRAINT:

```
1. CONSTRAINT FK_name FOREIGN KEY (column) REFERENCES ref_table(column)
```

Per poi eliminare una FOREIGN KEY bisogna fare:

```
1. ALTER TABLE table_name
2. DROP FOREIGN KEY FK_name;
```

SQL: CONSTRAINT

CHECK esempio:

```
1. CREATE TABLE Persons (
2.     ID int NOT NULL,
3.     LastName varchar(255) NOT NULL,
4.     FirstName varchar(255),
5.     Age int,
6.     CHECK (Age>=18) );
```

È possibile indicare un CHECK con un CONSTRAINT:

```
1. CONSTRAINT CHK_name CHECK (con1 AND con2)
```

Per poi eliminare un CHECK bisogna fare:

```
1. ALTER TABLE table_name
2. DROP CHECK CHK_name;
```

SQL: CONSTRAINT

DEFAULT esempio:

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    City varchar(255) DEFAULT 'Sandnes' );
```

È possibile indicare un valore con una function:

```
OrderDate date DEFAULT CURRENT_DATE()
```

Per poi eliminare o modificare un DEFAULT bisogna fare:

```
ALTER TABLE table_name
ALTER column DROP DEFAULT; / ALTER column SET DEFAULT value;
```

SQL: CONSTRAINT

AUTO_INCREMENT esempio:

```
1. CREATE TABLE Persons (
2.     Personid int NOT NULL AUTO_INCREMENT,
3.     LastName varchar(255) NOT NULL,
4.     FirstName varchar(255),
5.     Age int,
6.     PRIMARY KEY (Personid) );
```

È possibile indicare un valore con cui far iniziare la sequenza:

```
1. ALTER TABLE Persons AUTO_INCREMENT=100;
```

NOTE: la colonna con il valore AUTO_INCREMENT non permette di inserirgli valori.

SQL: DATE

MySQL fornisce diversi tipi di DATE per la memorizzazione di una data o di un valore di data / ora nel database:

- **DATE - formato AAAA-MM-DD ('2038-01-19')**
- **DATETIME - formato: AAAA-MM-GG HH:MI:SS ('2038-01-19 03:14:07.999999')**
- **TIMESTAMP - formato: AAAA-MM-GG HH:MI:SS ('2038-01-19 03:14:07.999999')**
- **YEAR - formato AAAA o YY ('2038', '00' - '69' per il 2000, '70' - '99' per il 1900)**

NOTE: è possibile confrontare con operatori di uguaglianza solo DATE senza valori temporali (data, ora e minuti).

SQL: VIEW

Una VIEW è un tabella virtuale basata sul result-set di una QUERY, contiene righe e colonne come una tabella. I campi di una VIEW sono campi di una o più tabelle presenti nel database.

```
1. CREATE VIEW view_name AS
2. SELECT column1, column2, ...
3. FROM table_name
4. WHERE condition;
```

Nota: una VIEW mostra sempre dati aggiornati, il di database ricrea la VIEW ad ogni QUERY.

```
1. CREATE VIEW [Brazil Customers] AS
2. SELECT CustomerName, ContactName
3. FROM Customers
4. WHERE Country = 'Brazil';
```

SQL: VIEW

Una VIEW si può aggiornare modificandola con il comando CREATE OR REPLACE VIEW:

```
1. CREATE OR REPLACE VIEW view_name AS
2. SELECT column1, column2, ...
3. FROM table_name
4. WHERE condition;
```

Per eliminare una VIEW invece bisogna usare DROP VIEW:

```
1. DROP VIEW view_name;
```

Esercizio:

Creare una view di city del database world su una query che mostra le città italiane.

Su questa VIEW eseguire una query che mostra solo le città con una popolazione inferiore ai 100k.

SQL in Java

JDBC:

In informatica JDBC (Java DataBase Connectivity), è un connettore e un driver per database che consente l'accesso e la gestione della persistenza dei dati sulle basi di dati da qualsiasi programma scritto con il linguaggio di programmazione Java, indipendentemente dal tipo di DBMS utilizzato.

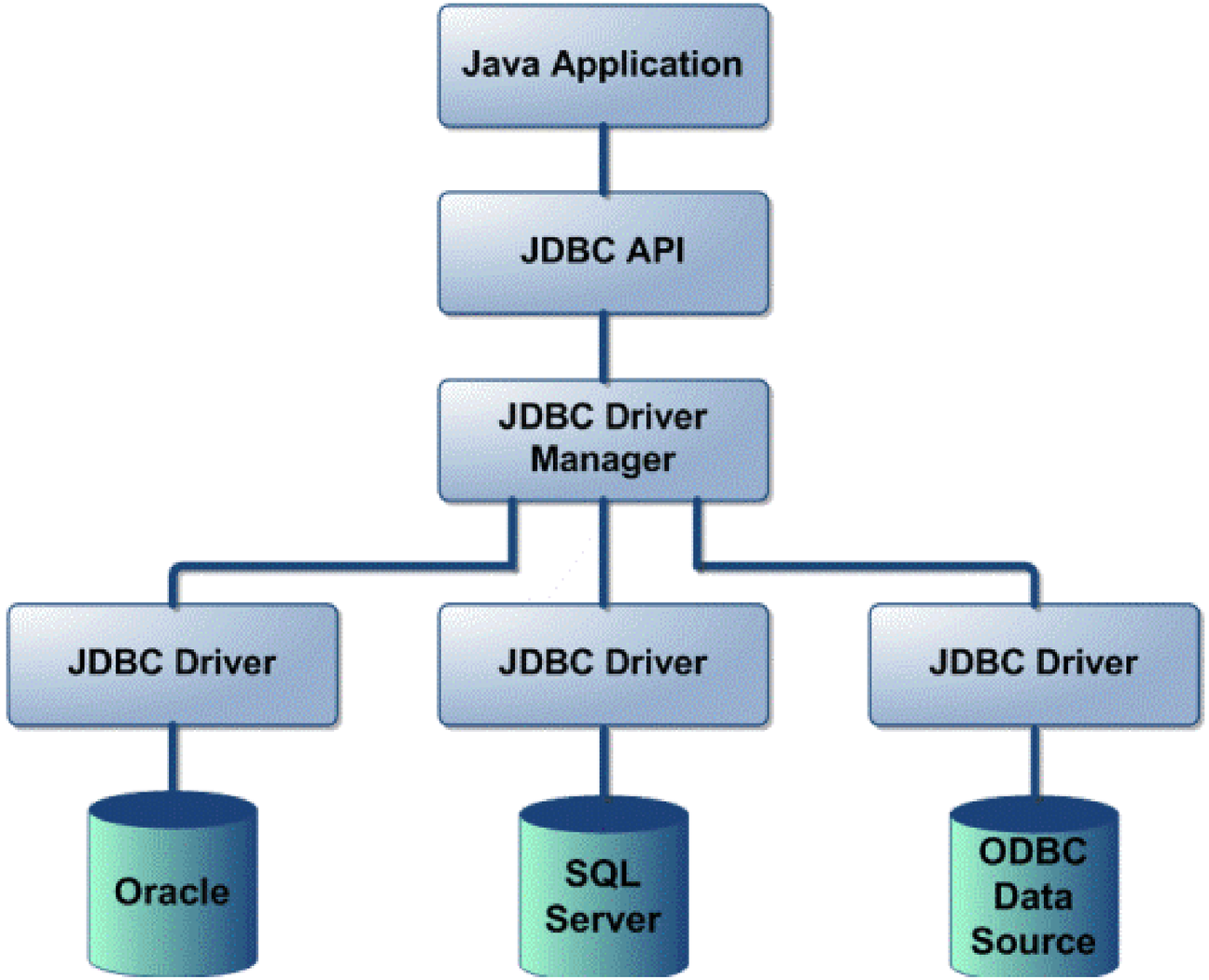
È costituito da un'API object oriented orientata ai database relazionali, raggruppata nel package java.sql, che serve ai client per connettersi a un database fornendo i metodi per interrogare e modificare i dati.

L'architettura di JDBC, prevede l'utilizzo di un “driver manager”, che espone alle applicazioni un insieme di interfacce standard e si occupa di caricare a “run-time” i driver opportuni per “pilotare” gli specifici DBMS.

Le applicazioni Java utilizzano le "JDBC API" per parlare con il JDBC driver manager, mentre il driver manager usa le JDBC driver API per parlare con i singoli driver che pilotano i DBMS specifici.

JDBC

NOTA: ODBC è un driver attraverso un'API standard per la connessione dal client al DBMS



JDBC:

L'API JDBC è costituita dai seguenti componenti principali:

- **JDBC Drivers: una raccolta di classi e interfacce per comunicare con il database;**
- **Connections: stabilisce una connessione con il database;**
- **Statements: dichiarazioni che servono per avviare le QUERY;**
- **ResultSets: gli elementi restituiti da una QUERY, La classe ResultSet fornisce un cursore che punta alla riga corrente nel result-set fornito dalla query;**

I passaggi per connettere il database utilizzando JDBC sono:

1. **Caricare il driver JDBC;**
2. **Connessione;**
3. **Statements;**
4. **Esecuzione statements;**
5. **Chiudere la connessione al database;**

JDBC: DRIVERS & CONNECTION

I driver da caricare cambiano in base al DBMS che utiliziamo.

Vengono caricati a runTime grazie alla classe DriverManager con l'url di connessione al DB formattato secondo lo standard jdbc e varia in base al DBMS.

In MySQL l'url è: jdbc:mysql://localhost:3306/nomeSchema
Possiamo scomporlo in 3 punti:

1. **jdbc:mysql:// >> prefisso standard per le connessioni MySQL;**
2. **localhost:3306 >> nome del server e porta che ospita il DB;**
3. **nomeSchema >> nome dello schema a cui fare riferimento;**

Una volta recuperato l'url di connessione è possibile effettuare la connessione al DB con le credenziali di accesso al DB.

Questo ci permetterà di ricavare l'oggetto Connection grazie al DriverManager:

```
1. Connection conn = DriverManager.getConnection(url,username,password);
```

JDBC: DRIVERS & CONNECTION

Per far funzionare la connessione abbiamo bisogno di importare il Driver "mysql-connector-j" per MySql nel progetto, questo permetterà al DriverManager di recuperare il driver necessario per la comunicazione con il DB.

Potrebbe essere necessario chiamare manualmente la registrazione del driver:

```
1. String DB_DRIVER = "com.mysql.jdbc.Driver";
2. Class.forName(DB_DRIVER);
```

Per utilizzare invece il DriverManager e Connection abbiamo bisogno di importare due librerie:

```
1. import java.sql.Connection;
2. import java.sql.DriverManager;
```

JDBC: DRIVERS & CONNECTION

Un esempio di sintassi con anche chiamata manuale del Driver:

```
1. String DB_DRIVER = "com.mysql.jdbc.Driver";
2. String DB_URL = "jdbc:mysql://localhost:3306/world";
3. String DB_USERNAME = "root";
4. String DB_PASSWORD = "root";
5. Connection conn = null;
6. try {
7.         // Register the JDBC driver
8.         Class.forName(DB_DRIVER);
9.         // Open the connection
10.        conn = DriverManager.getConnection(DB_URL, DB_USERNAME, DB_PASSWORD);
11.         if (conn != null)
12.            System.out.println("Successfully connected.");
13.         else
14.            System.out.println("Failed to connect.");
15.        } catch (Exception e) {
16.            e.printStackTrace(); }
```

JDBC: STATEMENT & RESULT-SET

Le interfacce Statement definiscono i metodi e le proprietà che consentono di inviare comandi SQL e ricevere dati dal database. Ci sono tre tipi di statement:

- **Statement - non è in grado di accettare parametri;**
- **PreparedStatement - accetta parametri di input;**
- **CallableStatement - accetta parametri di input a runtime;**

La differenza principale tra Prepared e Callable è che PreparedStatement viene gestito in Java mentre CallableStatement viene recuperato dal database.

Quindi Statement viene usato per query fisse, statiche. Invece PreparedStatement ci permette di settare dei valori prima di eseguire la query.

```
1. Statement stmt =  conn.createStatement();
2. ResultSet rs = stmt.executeQuery(QUERY);
```

JDBC: STATEMENT & RESULT-SET

Una volta creato lo statement bisogna eseguirlo per recuperare un oggetto di ResultSet:

```
1. PreparedStatement stmt =  conn.prepareStatement(QUERY);
2. ResultSet rs = stmt.executeQuery();
```

Una volta recuperato il result-set è possibile accedervi come se fosse una matrice sfruttando un ciclo while per far avanzare il cursore del result-set su tutte le righe:

```
1. while(rs.next())
2.     {
3.       System.out.println(rs.getString(1)); //First Column
4.       System.out.println(rs.getString(2)); //Second Column
5.       System.out.println(rs.getString(3)); //Third Column
6.       System.out.println(rs.getString(4)); //Fourth Column
7.     }
```

JDBC: RESULT-SET

È possibile anche indicare il nome della colonna nei getter dei valori per le colonne:

```
1. System.out.println(rs.getString("colomn_name"));
```

È possibile recuperare anche i METADATI dal result-set, ovvero tutte le informazioni riguardanti la struttura della tabella come ad esempio il tipo di una colonna o il numero di colonne o il nome della tabella ecc ecc...

```
1. ResultSetMetaData metaData = rs.getMetaData();
2. Integer columnCount = metaData.getColumnCount();
```

Nell'esempio andiamo a recuperare il numero di colonne del result-set.

NOTA: è necessaria la libreria java.sql.ResultSetMetaData.

JDBC: RESULT-SET

Le principali funzioni della classe ResultSetMetaData sono:

- **getColumnCount() - restituisce il numero di colonne;**
- **getColumnName(int columnNumber) – restituisce il nome della colonna;**
- **getColumnLabel(int columnNumber) – restituisce l'alias assegnato nella QUERY alla colonna;**
- **getTableName(int columnNumber) – restituisce il nome della tabella;**
- **getColumnType(int columnNumber) – restituisce il tipo di dato della colonna;**
- **isAutoIncrement(int columnNumber) – indica se la colonna è auto-incrementata;**
- **isCaseSensitive(int columnNumber) – indica se la colonna è case sensitive;**
- **isSearchable(int columnNumber) – indica se possiamo usare la colonna nel WHERE;**
- **isNullable(int columnNumber):**
  - **restituisce:**
    - **0 se può avere NULL;**
    - **1 se non può avere NULL;**
    - **2 se è sconosciuto;**

JDBC: RESULT-SET

Il result-set ottenuto è navigabile grazie ad un cursore, che di default avanza alle righe successive.

La navigazione del cursore è modificabile tramite dei valori da passare al momento della creazione dello statement (se è supportato dal DBMS).

Possiamo usare diverse funzioni per far scorrere il cursore:

- **next() – passa alla riga successiva;**
- **previous() – passa alla riga precedente;**
- **first() – passa alla prima riga del ResultSet;**
- **last() – passa all'ultima riga;**
- **beforeFirst() – si sposta all'inizio, prima della prima riga;**
- **afterLast() – si sposta alla fine, dopo l'ultima riga;**
- **relative(int numOfRows) – si sposta del numero di righe indicate (anche in negativo);**
- **absolute(int rowNumber) – passa alla riga specificata;**
- **getRow() – recupera il numero della riga;**

JDBC: RESULT-SET

Di default il result-set è di sola lettura, per renderlo modificabile è necessario indicare alla creazione dello statement che può essere modificato:

```
1. PreparedStatement pstmt = dbConnection.prepareStatement(
2.   QUERY, // la query da eseguire
3.   ResultSet.TYPE_SCROLL_SENSITIVE, // indichiamo il tipo di scorrimento del cursore
4.   ResultSet.CONCUR_UPDATABLE); // indichiamo che il result-set è aggiornabile
5. ResultSet rs = pstmt.executeQuery();
```

Si possono inserire e modificare i record del result-set ottenuto:

```
1. rs.updateDouble("salary", 1100.0);
2.  // possiamo passare anche il numero della colonna
3. rs.updateRow();
4. // aggiorniamo il database
```

JDBC: RESULT-SET

Per aggiungere una o più righe dobbiamo prima spostarci con il cursore:

```
1. rs.moveToInsertRow(); // sposta il cursore per inserire una riga
```

Ora possiamo aggiornare i valori della nuova riga:

```
1. rs.updateString("name", "Venkat"); //aggiorna il tipo string
2. rs.updateString("position", "DBA"); //aggiorna il tipo string
3. rs.updateDouble("salary", 925.0); // aggiorna il tipo double
```

Infine possiamo inserire la nuova riga al database:

```
1. rs.insertRow(); // aggiorna il database con la nuova riga
```

Adesso possiamo spostarci alla riga a cui ci trovavamo prima del metodo moveToInsertRow():

```
1. rs.moveToCurrentRow(); // sposta il cursore a prima di moveToInsertRow()
```

JDBC: RESULT-SET

Per rimuovere una riga invece bisogna posizionarsi con il cursore alla riga interessata e poi eliminarla:

```
1. rs.absolute(2); // sposta il cursore alla riga indicata
2. rs.deleteRow();
```

Il recupero dei record di un result-set da una QUERY avviene tutto contemporaneamente in memoria.
Possiamo indicare il numero di righe massimo da caricare in memoria contemporaneamente:

```
1. PreparedStatement pstmt = dbConnection.prepareStatement(QUERY);
2. pstmt.setFetchSize(10);
3. ResultSet rs = pstmt.executeQuery();
```

NOTA: ad esempio un risultato di 100 righe verrà visualizzato con 10 cicli da 10 righe di lettura al database. È possibile usare la stessa istruzione per indicare un limite di memoria al result-set.

Esercizio:
Usando jdbc recuperare e stampare una view che mostri le città italiane presenti nel database world, stampando la view sfruttando le informazioni ricavate dai metadati (quindi con un metodo di stampa generalizzato).

Esercizio:
Aggiungere tramite JDBC 10 città italiane non presenti nella tabella city di world con anche inserendo i corrispettivi dati.
Da svolgere senza le query ma usando i metodi del ResultSet (JDBC).

SQL in Python

sqlite3:

- **SQLite è un database che vive in un unico file (es. scuola.db): niente server da installare o avviare.**
- **sqlite3 è il modulo Python per usarlo: è già incluso in Python, basta import sqlite3.**
- **Con sqlite3 scriviamo i comandi SQL che già conosciamo (CREATE, INSERT, SELECT, UPDATE, DELETE) dentro un programma Python.**
- **Il flusso è sempre lo stesso:**
  a. **connect → apro (o creo) il file del database**
  b. **cursor → creo il "cursore", l'oggetto che esegue i comandi**
  c. **execute → eseguo un comando SQL (scritto come stringa)**
  d. **commit → salvo le modifiche (solo per INSERT, UPDATE, DELETE)**
  e. **fetch → leggo i risultati (solo per SELECT)**
  f. **close → chiudo la connessione**
- **I risultati di una SELECT arrivano come tuple Python: una tupla per ogni riga.**
- **Per inserire valori presi da variabili si usa il segnaposto ?, mai f-string o concatenazione: è più sicuro e gestisce da solo virgolette e tipi.**
- **conn e cur fanno due lavori diversi: i comandi passano dal cursore (execute, fetchall), il salvataggio e la chiusura passano dalla connessione (commit, close).**

Esempi di utilizzo

- **Salvare dati tra un'esecuzione e l'altra: una lista Python sparisce quando il programma termina, una tabella nel file .db resta.**
- **Piccoli gestionali: rubrica, magazzino di un negozio, registro voti, prestiti di una biblioteca.**
- **Programmi interattivi: l'utente inserisce dati con input() e il programma li salva o li cerca nel database.**
- **Analisi dati: uso SQL per filtrare e ordinare (WHERE, ORDER BY, AVG...) e Python per elaborare o stampare i risultati.**
- **Prototipi e prove: prima di passare a database "grandi" (MySQL, PostgreSQL), si sperimenta con SQLite perché è immediato.**
- **App e giochi: molte app mobili e giochi salvano punteggi e impostazioni proprio in un file SQLite.**

sqlite3 Types

Tipo	Descrizione breve	Spazio occupato	Esempio (colonna → valore)
NULL	Valore assente	0 byte di dati	telefono → NULL
INTEGER	Numero intero con segno	0, 1, 2, 3, 4, 6 o 8 byte, in base al valore (da -128 a 127 → 1 byte; fino a ±9.223.372.036.854.775.807 → 8 byte). Per i valori 0 e 1 non servono byte di dati	età INTEGER → 25
REAL	Decimale in virgola mobile a 64 bit	Sempre 8 byte	prezzo REAL → 1.35
TEXT	Testo	Un byte per carattere semplice, 2 per le lettere accentate (UTF-8)	nome TEXT → 'Anna'
BLOB	Dati binari salvati così come sono (immagini, file)	Tanti byte quanti sono i dati	foto BLOB → X'89504E47'

Tipi “finti” molto usati

Nome	Come viene salvato	Spazio occupato	Esempio
BOOLEAN	Come intero: vero = 1, falso = 0	Come INTEGER	attivo BOOLEAN → 1
DATE / DATETIME	Come testo, numero (tempo Unix) o REAL: la scelta è tua	Come testo: 10 byte per 'AAAA-MM-GG', 19 con l’ora	data DATE → '2026-10-16'

Come SQLite interpreta il tipo che dichiari

Tipo dichiarato	Viene trattato come	Cosa succede	Tipo dichiarato
INT, INTEGER, TINYINT, SMALLINT, MEDIUMINT, BIGINT, INT2, INT8, UNSIGNED BIG INT	INTEGER	Il testo '12' diventa il numero 12, 'abc' resta testo	INT, INTEGER, TINYINT, SMALLINT, MEDIUMINT, BIGINT, INT2, INT8, UNSIGNED BIG INT
CHARACTER(n), VARCHAR(n), NCHAR(n), NVARCHAR(n), TEXT, CLOB	TEXT	Tutto resta testo, anche '12'. La lunghezza (n) viene ignorata	CHARACTER(n), VARCHAR(n), NCHAR(n), NVARCHAR(n), TEXT, CLOB
REAL, DOUBLE, DOUBLE PRECISION, FLOAT	REAL	'12' diventa 12.0	REAL, DOUBLE, DOUBLE PRECISION, FLOAT
BLOB oppure nessun tipo	Nessuna conversione	Il valore resta com'è stato inserito	BLOB oppure nessun tipo
NUMERIC, DECIMAL(p,s), BOOLEAN, DATE, DATETIME e qualsiasi altro nome (es. STRING, JSON)	NUMERIC	Se il testo sembra un numero diventa intero o decimale ('3.0' diventa 3), altrimenti resta testo	NUMERIC, DECIMAL(p,s), BOOLEAN, DATE, DATETIME e qualsiasi altro nome (es. STRING, JSON)

!!! ATTENZIONE: Interi troppo grandi: da Python un intero oltre gli 8 byte dà OverflowError.

Esempio

Esempio

Cosa mostra: creare il database, una tabella e inserire una riga.

```
import sqlite3  # modulo già incluso in Python

conn = sqlite3.connect("scuola.db")  # apre il file scuola.db (lo crea se non esiste)
cur = conn.cursor()                  # il cursore è l'oggetto che esegue i comandi SQL

cur.execute("""
    CREATE TABLE IF NOT EXISTS studenti (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nome TEXT NOT NULL,
        voto INTEGER
    )
""")  # crea la tabella solo se non esiste già

cur.execute("INSERT INTO studenti (nome, voto) VALUES ('Anna', 8)")  # inserisce una riga
conn.commit()  # salva davvero le modifiche nel file
conn.close()   # chiude la connessione
```

- Il comando SQL è una normale stringa Python passata a execute().
- Le triple virgolette """ permettono di scrivere SQL su più righe.
- IF NOT EXISTS evita l'errore se lanciamo il programma una seconda volta.
- Senza commit() l'INSERT non viene salvato nel file.

Esempio

Esempio

Cosa mostra: creare il database, una tabella e inserire una riga.

```
import sqlite3  # modulo già incluso in Python

conn = sqlite3.connect("scuola.db")  # apre il file scuola.db (lo crea se non esiste)
cur = conn.cursor()                  # il cursore è l'oggetto che esegue i comandi SQL

cur.execute("""
    CREATE TABLE IF NOT EXISTS studenti (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nome TEXT NOT NULL,
        voto INTEGER
    )
""")  # crea la tabella solo se non esiste già

cur.execute("INSERT INTO studenti (nome, voto) VALUES ('Anna', 8)")  # inserisce una riga
conn.commit()  # salva davvero le modifiche nel file
conn.close()   # chiude la connessione
```

Attenzione – righe duplicate

- Se rilanci il programma, Anna viene inserita di nuovo: IF NOT EXISTS protegge la tabella, non le righe.
- Per ripartire da zero durante le prove: cancella il file scuola.db oppure esegui DELETE FROM studenti.
- Il file .db viene creato nella cartella da cui lanci il programma. ~~Per vederne il contenuto senza codice puoi usare DB Browser for SQLite (gratuito).~~

Esempio

Cosa mostra: inserire più righe da variabili Python e leggerle con una SELECT.

```
import sqlite3

conn = sqlite3.connect("scuola.db")
cur = conn.cursor()

nuovi = [("Luca", 6), ("Sara", 9), ("Marco", 5)]  # lista di tuple: una tupla = una riga
cur.executemany("INSERT INTO studenti (nome, voto) VALUES (?, ?)", nuovi)  # ogni ? viene riempito con un valore della tupla
conn.commit()  # salvo gli inserimenti

cur.execute("SELECT nome, voto FROM studenti WHERE voto >= ?", (6,))  # anche un solo valore va in una tupla: (6,)
risultati = cur.fetchall()  # lista di tuple, es. [('Anna', 8), ('Luca', 6), ('Sara', 9)]

for nome, voto in risultati:  # "spacchetto" ogni tupla in due variabili
    print(nome, "ha preso", voto)

conn.close()
```

- executemany() esegue lo stesso INSERT una volta per ogni tupla della lista.
- I ? sono segnaposto: i valori vengono inseriti nell'ordine in cui compaiono.
- fetchall() restituisce tutte le righe trovate come lista di tuple.
- La SELECT non modifica nulla, quindi non serve commit().

Esempio

Esempio

Perché i ? e non le f-string? Basta un apostrofo per rompere tutto:

```
nome = "D'Angelo"
cur.execute(f"INSERT INTO studenti (nome, voto) VALUES ('{nome}', 7)")   # errore di sintassi SQL
cur.execute("INSERT INTO studenti (nome, voto) VALUES (?, ?)", (nome, 7))  # funziona
```

I valori per i ? vanno sempre in una tupla, anche se è uno solo: (6,) con la virgola. (6) per Python è solo il numero 6.

Esempio

Variante – funzioni di aggregazione Cosa mostra: usare COUNT e AVG da Python e leggere il risultato.

```
cur.execute("SELECT COUNT(*), AVG(voto) FROM studenti")  # SQL già noto: conta e fa la media
totale, media = cur.fetchone()  # una sola riga → una tupla, es. (4, 7.0)
print("Studenti:", totale, "- Media voti:", media)
```

- Anche un'aggregazione restituisce una riga, quindi si usa fetchone().
- Con un solo valore si può scrivere cur.fetchone()[0].

Esempio

Cosa mostra: una funzione che cerca uno studente scelto dall'utente e ne aggiorna il voto.

```
import sqlite3

def aggiorna_voto(nome, nuovo_voto):
    conn = sqlite3.connect("scuola.db")
    cur = conn.cursor()

    cur.execute("SELECT voto FROM studenti WHERE nome = ?", (nome,))  # cerco lo studente
    riga = cur.fetchone()  # una sola riga (tupla) oppure None se non c'è

    if riga is None:  # nessuno studente con quel nome
        print(nome, "non trovato")
    else:
        vecchio_voto = riga[0]  # il primo (e unico) valore della tupla
        cur.execute("UPDATE studenti SET voto = ? WHERE nome = ?", (nuovo_voto, nome))  # due ? → tupla con due valori
        conn.commit()  # UPDATE modifica i dati: serve il commit
        print(nome, ":", vecchio_voto, "→", nuovo_voto)

    conn.close()

nome = input("Nome studente: ")      # il valore arriva dall'utente...
voto = int(input("Nuovo voto: "))    # ...e lo convertiamo in intero
aggiorna_voto(nome, voto)            # ...e lo passiamo in sicurezza con i ?
```

- **fetchone() restituisce una sola riga oppure None: utile per controllare se un dato esiste.**
- **Prima si controlla con una SELECT, poi si modifica con l'UPDATE.**
- **Anche se il nome arriva da input(), grazie ai ? è sempre trattato come semplice valore, mai come codice SQL.**
- **Raccogliere le operazioni in una funzione rende il codice riutilizzabile.**

Gli errori più comuni

Messaggio	Causa tipica	Soluzione
no such table: studenti	tabella non creata, nome sbagliato o file .db diverso	controlla il CREATE TABLE e la cartella di lavoro
near "SELCT": syntax error	errore di battitura nel comando SQL	rileggi la stringa SQL parola per parola
Incorrect number of bindings supplied	numero di ? diverso dai valori nella tupla	conta i ? e i valori; ricorda la virgola in (x,)

Esercizi

- **Es1**
  - **Obiettivo:** crea un database rubrica.db con una tabella contatti (id, nome, telefono). Inserisci 3 contatti a tua scelta, salva le modifiche e stampa a video tutti i contatti presenti nella tabella.

- **Es2**
  - **Obiettivo:** crea un database negozio.db con una tabella prodotti (id, nome, prezzo, quantita). Inserisci almeno 5 prodotti partendo da una lista di tuple. Poi chiedi all'utente un prezzo massimo con input() e stampa solo i prodotti che costano meno di quel prezzo, ordinati dal più economico al più caro.

- **Es3**
  - **Obiettivo:** crea un programma per gestire una collezione di film (film.db, tabella film con id, titolo, anno, voto da 1 a 10). Il programma mostra un menu che si ripete finché l'utente non sceglie di uscire:
    - aggiungi un film
    - mostra i film di un anno scelto dall'utente
    - elimina un film dato il titolo (avvisando se il film non esiste)
    - esci
  - **Ogni operazione deve stare in una funzione separata.**

UPDATE, DELETE e gestione degli errori con sqlite3:

- **Il flusso è lo stesso già visto: connect → cursor → execute → commit → close. Cambiano solo i comandi SQL: ora usiamo UPDATE (modifica) e DELETE (cancella).**
- **UPDATE e DELETE vanno sempre con una clausola WHERE: senza, il comando colpisce tutte le righe della tabella.**
- **Anche qui i valori si passano con i segnaposto ?, dentro una tupla.**
- **Dopo UPDATE e DELETE serve commit(), altrimenti le modifiche non vengono salvate nel file.**
- **cur.rowcount dice quante righe sono state toccate dall'ultimo comando:**
  - **1 (o più) → il comando ha trovato qualcosa da modificare/cancellare**
  - **0 → nessuna riga corrisponde al WHERE (nessun errore, ma nulla è cambiato)**

UPDATE, DELETE e gestione degli errori con sqlite3:

- **with conn: gestisce il salvataggio al posto nostro: se tutto va bene fa commit, se c'è un errore fa rollback (annulla le modifiche non salvate). Attenzione: non chiude la connessione.**
- **with conn: o commit() esplicito? Per le operazioni semplici sono equivalenti, ma with è più comodo e più sicuro. Attenzione: se c'è un errore, with fa il rollback ma rilancia l'errore: per non far crashare il programma serve comunque un try / except intorno.**
- **try / except sqlite3.Error permette di intercettare gli errori del database senza far "crashare" il programma. Un caso tipico è sqlite3.IntegrityError, che scatta quando si viola un vincolo (valore duplicato in una chiave, campo NOT NULL vuoto, ecc.).**
- **Con finally si scrive il codice che deve essere eseguito sempre, anche dopo un errore: tipicamente conn.close().**

Esempio

Cosa mostra: modificare e cancellare righe con UPDATE e DELETE, controllando rowcount.

```
import sqlite3

conn = sqlite3.connect("azienda.db")
cur = conn.cursor()

cur.execute("DROP TABLE IF EXISTS dipendenti")  # ripartiamo da zero a ogni prova
cur.execute("""
   CREATE TABLE dipendenti (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      nome TEXT NOT NULL,
      reparto TEXT,
      stipendio INTEGER
   )
""")
cur.executemany(
   "INSERT INTO dipendenti (nome, reparto, stipendio) VALUES (?, ?, ?)",
   [("Anna", "Vendite", 1500), ("Luca", "Vendite", 1400), ("Sara", "Magazzino", 1300)]
)
conn.commit()  # salvo i dati di partenza

cur.execute("UPDATE dipendenti SET stipendio = ? WHERE nome = ?", (1600, "Anna"))  # modifica solo Anna
print("Righe modificate:", cur.rowcount)  # 1 → Anna è stata trovata

cur.execute("DELETE FROM dipendenti WHERE nome = ?", ("Luca",))  # cancella solo Luca
print("Righe cancellate:", cur.rowcount)  # 1

cur.execute("DELETE FROM dipendenti WHERE nome = ?", ("Zeno",))  # Zeno non esiste
print("Righe cancellate:", cur.rowcount)  # 0 → nessun errore, ma nessuna riga toccata

conn.commit()  # salvo UPDATE e DELETE
conn.close()
```

- **UPDATE ... SET ... WHERE ... cambia solo le righe che rispettano la condizione.**
- **rowcount ci dice se il WHERE ha trovato qualcosa: 0 non è un errore, è un'informazione.**
- **Senza il WHERE l'UPDATE cambierebbe lo stipendio di tutti i dipendenti.**
- **Il commit() finale salva entrambe le modifiche nel file azienda.db.**

Esempio

Cosa mostra: usare with conn: e intercettare un errore di valore duplicato con IntegrityError.

```
import sqlite3

conn = sqlite3.connect("azienda.db")
cur = conn.cursor()

cur.execute("DROP TABLE IF EXISTS badge")
cur.execute("""
   CREATE TABLE badge (
      codice TEXT PRIMARY KEY,   -- il codice deve essere unico
      dipendente TEXT NOT NULL
   )
""")
conn.commit()

def assegna_badge(codice, dipendente):
   try:
      with conn:  # commit automatico se va bene, rollback automatico se c'è un errore
         cur.execute("INSERT INTO badge (codice, dipendente) VALUES (?, ?)", (codice, dipendente))
      print("Badge", codice, "assegnato a", dipendente)
   except sqlite3.IntegrityError:  # scatta se il codice esiste già
      print("Errore: il badge", codice, "è già stato assegnato")

assegna_badge("B01", "Anna")
assegna_badge("B02", "Luca")
assegna_badge("B01", "Sara")  # codice duplicato → viene gestito dall'except

conn.close()
```

- **Il PRIMARY KEY impedisce due righe con lo stesso codice: il secondo inserimento genera un IntegrityError.**
- **Dentro with conn: non serve scrivere commit(): lo fa Python se non ci sono errori.**
- **L'except intercetta l'errore e mostra un messaggio chiaro: il programma non si interrompe.**
- **Il with non chiude la connessione, per questo conn.close() resta in fondo.**

Esempio

Cosa mostra: una funzione completa con controlli, rollback esplicito e finally.

```
import sqlite3

def aggiorna_stipendio(id_dip, nuovo):
    conn = sqlite3.connect("azienda.db")
    cur = conn.cursor()
    try:
        if nuovo < 0:
            raise ValueError("lo stipendio non può essere negativo")  # errore nostro, prima di toccare il database

        cur.execute("UPDATE dipendenti SET stipendio = ? WHERE id = ?", (nuovo, id_dip))
        if cur.rowcount == 0:  # nessuna riga trovata: non c'è nulla da salvare
            print("Nessun dipendente con id", id_dip)
        else:
            conn.commit()
            print("Stipendio aggiornato")
    except ValueError as errore:
        print("Dato non valido:", errore)
    except sqlite3.Error as errore:  # qualunque altro errore del database
        conn.rollback()  # annullo le modifiche non ancora salvate
        print("Errore del database:", errore)
    finally:
        conn.close()  # viene eseguito SEMPRE, anche dopo un errore

aggiorna_stipendio(1, 1700)   # ok
aggiorna_stipendio(99, 1000)  # id inesistente → rowcount = 0
aggiorna_stipendio(1, -50)    # dato non valido → ValueError
```

- La funzione apre e chiude la propria connessione: ogni chiamata è indipendente.
- rowcount == 0 evita di dire "fatto!" quando l'id non esiste.
- I due except distinguono un dato sbagliato (nostro) da un problema del database.
- finally garantisce la chiusura della connessione in tutti i casi.

Esempio

Variante – due modifiche collegate: il rollback in azione Cosa mostra: due UPDATE che devono riuscire insieme; se la seconda fallisce, la prima viene annullata.

```
import sqlite3

conn = sqlite3.connect("azienda.db")
cur = conn.cursor()

cur.execute("SELECT stipendio FROM dipendenti WHERE id = 1")
print("Stipendio di Anna prima:", cur.fetchone()[0])

try:
    cur.execute("UPDATE dipendenti SET stipendio = stipendio - 100 WHERE id = ?", (1,))   # prima modifica: va a buon fine
    cur.execute("UPDATE dipendenti SET stipendio = stipendio + 100 WHERE id = ?", (99,))  # seconda modifica: l'id 99 non esiste
    if cur.rowcount == 0:  # nessuna riga modificata dalla seconda UPDATE
        raise ValueError("secondo dipendente non trovato")
    conn.commit()  # si arriva qui solo se è andato tutto bene
    print("Spostamento completato")
except (ValueError, sqlite3.Error) as errore:
    conn.rollback()  # annulla ANCHE la prima UPDATE, che non era ancora stata salvata
    print("Operazione annullata:", errore)

cur.execute("SELECT stipendio FROM dipendenti WHERE id = 1")
print("Stipendio di Anna dopo:", cur.fetchone()[0])  # uguale a prima
conn.close()
```

- **Tra le due UPDATE non c'è nessun commit(): finché non salviamo, il database può ancora tornare indietro.**
- **Il commit() è l'ultima istruzione del try: ci si arriva solo se non ci sono stati errori.**
- **rollback() annulla tutte le modifiche fatte dopo l'ultimo commit(), anche la prima UPDATE andata a buon fine.**
- **Senza il rollback, la modifica "a metà" potrebbe essere salvata da un commit() successivo.**

Esercizi

- **Es1**
  - **Obiettivo:** nel database palestra.db crea la tabella iscritti (id, nome, scadenza, con scadenza come testo, es. "2026-12-31") e inserisci 4 iscritti. Poi aggiorna la scadenza di un iscritto e cancella un altro iscritto. Dopo ogni operazione stampa quante righe sono state toccate (rowcount).

- **Es2**
  - **Obiettivo:** nel database ristorante.db gestisci una tabella prenotazioni (id, cliente, tavolo, ora) in cui la stessa coppia tavolo + ora non può comparire due volte (vincolo UNIQUE (tavolo, ora)). Scrivi due funzioni: prenota(cliente, tavolo, ora), che usa with conn: e mostra "Tavolo già occupato" se la coppia esiste già, e disdici(cliente), che cancella le prenotazioni di quel cliente e stampa "Nessuna prenotazione trovata" se rowcount è 0.

- **Es3**
  - **Obiettivo:** nel database banca.db crea la tabella conti (id, titolare, saldo) con almeno 3 conti. Scrivi la funzione bonifico(id_da, id_a, importo) che sposta il denaro con due UPDATE nella stessa operazione: se il saldo del conto di partenza non basta, oppure uno dei due conti non esiste, nessuna delle due modifiche deve restare salvata. Verifica con una chiamata riuscita e due chiamate che falliscono, stampando i saldi prima e dopo ogni tentativo.

Due tabelle collegate in sqlite3: chiavi esterne, JOIN e row_factory

- Un database ben fatto non ripete i dati: invece di scrivere il nome della squadra accanto a ogni giocatore, si creano due tabelle (squadre e giocatori) e si collegano.
- Il collegamento è una chiave esterna (foreign key): una colonna della seconda tabella (squadra_id) che contiene l'id di una riga della prima.
- Ripasso JOIN: JOIN … ON unisce le righe delle due tabelle quando la condizione è vera (es. giocatori.squadra_id = squadre.id). Da Python si scrive come qualsiasi altra query, dentro execute().
- Attenzione: in SQLite le chiavi esterne non sono controllate di default. Serve eseguire PRAGMA foreign_keys = ON a ogni nuova connessione; da quel momento un inserimento con un squadra_id inesistente genera un IntegrityError.

Due tabelle collegate in sqlite3: chiavi esterne, JOIN e row_factory

- **Se due tabelle hanno colonne con lo stesso nome (es. nome), si usano gli alias con AS: squadre.nome AS squadra.**
- **Per default i risultati sono tuple: leggerle con gli indici (riga[0], riga[2]) è poco chiaro. Con conn.row_factory = sqlite3.Row ogni riga si legge per nome di colonna: riga["squadra"].**
- **Dopo un INSERT, cur.lastrowid contiene l'id appena creato: utile per collegare subito altre righe.**

Esempio

cosa mostra: creare due tabelle collegate, inserire i dati e leggerli con un JOIN. Alla fine, un tentativo di inserimento che viola la chiave esterna.

```
import sqlite3

conn = sqlite3.connect("calcio.db")
cur = conn.cursor()
cur.execute("PRAGMA foreign_keys = ON")  # da rifare a ogni connessione, altrimenti le chiavi esterne non vengono controllate

cur.execute("DROP TABLE IF EXISTS giocatori")  # prima la tabella "figlia", poi la "madre"
cur.execute("DROP TABLE IF EXISTS squadre")
cur.execute("CREATE TABLE squadre (id INTEGER PRIMARY KEY AUTOINCREMENT, nome TEXT NOT NULL, citta TEXT)")
cur.execute("""
    CREATE TABLE giocatori (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nome TEXT NOT NULL,
        ruolo TEXT,
        squadra_id INTEGER,
        FOREIGN KEY (squadra_id) REFERENCES squadre(id)  -- squadra_id deve esistere in squadre
    )
""")

```

CONTINUA ->

- **FOREIGN KEY (squadra_id) REFERENCES squadre(id) dichiara il collegamento tra le due tabelle.**
- **Si cancellano prima le tabelle "figlie" e poi le "madri", altrimenti la chiave esterna lo impedisce.**

Esempio

```
cur.executemany("INSERT INTO squadre (nome, citta) VALUES (?, ?)", [("Aquile", "Torino"), ("Leoni", "Napoli")])
cur.executemany(
    "INSERT INTO giocatori (nome, ruolo, squadra_id) VALUES (?, ?, ?)",
    [("Marco", "portiere", 1), ("Elia", "attaccante", 1), ("Paolo", "difensore", 2)]
)
conn.commit()

cur.execute("""
    SELECT giocatori.nome, squadre.nome
    FROM giocatori
    JOIN squadre ON giocatori.squadra_id = squadre.id
""")
for giocatore, squadra in cur.fetchall():  # ogni riga è una tupla (nome giocatore, nome squadra)
    print(giocatore, "gioca per", squadra)

try:
    cur.execute("INSERT INTO giocatori (nome, ruolo, squadra_id) VALUES (?, ?, ?)", ("Fantasma", "attaccante", 99))
except sqlite3.IntegrityError as errore:  # la squadra 99 non esiste
    print("Errore:", errore)

conn.close()
```

- **Il JOIN ... ON abbina ogni giocatore alla sua squadra tramite squadra_id e id.**
- **Con PRAGMA foreign_keys = ON l'inserimento in una squadra inesistente fallisce con IntegrityError.**

Esempio

Attenzione – cancellare una squadra che ha ancora dei giocatori

```
import sqlite3

conn = sqlite3.connect("calcio.db")
cur = conn.cursor()
cur.execute("PRAGMA foreign_keys = ON")

# preparo una squadra di prova con un giocatore
cur.execute("INSERT INTO squadre (nome, citta) VALUES (?, ?)", ("Falchi", "Roma"))
id_falchi = cur.lastrowid  # id appena assegnato alla squadra
cur.execute("INSERT INTO giocatori (nome, ruolo, squadra_id) VALUES (?, ?, ?)", ("Prova", "portiere", id_falchi))
conn.commit()

try:
    cur.execute("DELETE FROM squadre WHERE id = ?", (id_falchi,))  # la squadra ha ancora un giocatore
    conn.commit()
except sqlite3.IntegrityError as errore:
    print("Impossibile cancellare:", errore)  # FOREIGN KEY constraint failed

cur.execute("DELETE FROM giocatori WHERE squadra_id = ?", (id_falchi,))  # prima i giocatori ("figli")
cur.execute("DELETE FROM squadre WHERE id = ?", (id_falchi,))          # poi la squadra ("madre")
conn.commit()
conn.close()
```

- Con le chiavi esterne attive non si può cancellare una riga "madre" finché ha righe "figlie" collegate.
- La regola pratica: si cancella prima chi dipende (i giocatori), poi chi è dipeso (la squadra).
- L'errore è un IntegrityError, quindi si può intercettare con try / except.

Esempio

Cosa mostra: leggere le colonne per nome con row_factory, usando alias e un parametro ? in un JOIN.

```
import sqlite3

conn = sqlite3.connect("calcio.db")
conn.row_factory = sqlite3.Row  # da ora le righe si leggono anche per nome di colonna
cur = conn.cursor()

cur.execute("""
    SELECT g.nome AS giocatore, g.ruolo, s.nome AS squadra   -- AS evita il conflitto tra i due "nome"
    FROM giocatori g
    JOIN squadre s ON g.squadra_id = s.id                    -- g e s sono nomi brevi (alias) delle tabelle
    WHERE s.citta = ?
""", ("Torino",))

for riga in cur.fetchall():
    print(riga["giocatore"], "-", riga["ruolo"], "-", riga["squadra"])  # lettura per nome, non per posizione

conn.close()
```

- **sqlite3.Row permette di scrivere riga["giocatore"] al posto di riga[0].**
- **AS assegna un nome alla colonna del risultato: serve quando due tabelle hanno colonne con lo stesso nome.**
- **g e s sono alias di tabella: rendono la query più corta e leggibile.**
- **Il parametro ? funziona anche nei JOIN, con la solita tupla (anche con un solo valore: ("Torino",)).**

Esempio

Cosa mostra: una funzione che cerca la squadra per nome, inserisce il giocatore collegato e usa lastrowid.

```
import sqlite3

def aggiungi_giocatore(nome, ruolo, nome_squadra):
    conn = sqlite3.connect("calcio.db")
    conn.row_factory = sqlite3.Row
    cur = conn.cursor()
    cur.execute("PRAGMA foreign_keys = ON")
    try:
        cur.execute("SELECT id FROM squadre WHERE nome = ?", (nome_squadra,))
        squadra = cur.fetchone()  # una riga, oppure None se la squadra non esiste
        if squadra is None:
            print("Squadra non trovata:", nome_squadra)
            return
        with conn:  # commit automatico, rollback se c'è un errore
            cur.execute(
                "INSERT INTO giocatori (nome, ruolo, squadra_id) VALUES (?, ?, ?)",
                (nome, ruolo, squadra["id"])  # uso l'id trovato, senza scriverlo a mano
            )
        print("Giocatore", nome, "inserito con id", cur.lastrowid)
    except sqlite3.Error as errore:
        print("Errore del database:", errore)
    finally:
        conn.close()
```

- **fetchone() restituisce None quando la SELECT non trova nulla: va controllato prima di leggere squadra["id"].**
- **Cerchiamo l'id con una SELECT invece di scriverlo a mano: l'utente conosce il nome, non il numero.**
- **cur.lastrowid è l'id assegnato alla riga appena inserita.**
- **return dentro il try esce dalla funzione, ma il finally chiude comunque la connessione.**

Esercizi

- **Es1**
  - **Obiettivo:** nel database **musica.db** crea le tabelle **artisti** (id, nome) e **album** (id, titolo, anno, artista_id collegato ad artisti). Inserisci 2 artisti e 4 album, poi stampa per ogni album una riga nel formato "titolo (anno) - artista", usando un **JOIN**.
    - Usa ***“PRAGMA foreign_keys = ON”*** e effettua un **INSERT INTO** con foreignkey non esistente gestendo l’errore con try-except.
    - Gestite anche un caso di **DELETE** di un artista collegato ad un album (sempre gestendo con try-except.

- **Es2**
  - **Obiettivo:** nel database **ordini.db** crea le tabelle clienti (id, nome, citta) e **ordini** (id, cliente_id, importo) con almeno 4 clienti e 8 ordini, attivando le chiavi esterne. Usando **row_factory**, stampa per ogni cliente il totale speso (suggerimento: SUM e GROUP BY). Poi ~~scrivi una funzione **ordini_di(nome_cliente)** che~~ stampa tutti gli ordini di quel cliente, oppure "Cliente non trovato" se il nome non esiste.

- **Es3**
  - **Obiettivo:** nel database **hotel.db** crea le tabelle **camere (id, numero, posti)** e **prenotazioni (id, camera_id, ospite, notti)**. Scrivi la funzione **prenota(numero_camera, ospite, notti)**, che cerca la camera dal suo numero, mostra un messaggio chiaro se non esiste e altrimenti inserisce la prenotazione. Scrivi poi **riepilogo()**, che stampa tutte le camere con le relative prenotazioni, includendo anche le camere ancora libere (suggerimento: LEFT JOIN).

Organizzare il codice: funzioni e menu con sqlite3

- **Quando un programma cresce, ripetere ovunque connect, cursor, commit, close diventa pesante e fonte di errori. La soluzione è organizzare il codice in funzioni: una funzione = un compito.**
- **Una funzione di accesso al database di solito:**
  - **apre la connessione (con una funzione dedicata, es. connetti())**
  - **esegue il codmando SQL**
  - **chiude la connessione**
  - **restituisce il risultato con return**
- **Scorciatoia: conn.execute(...) crea un cursore al volo e lo restituisce, equivale a cur = conn.cursor() seguito da cur.execute(...).**
- **Separare i compiti: le funzioni che parlano con il database restituiscono i dati (non stampano nulla); il menu e le funzioni di stampa si occupano di mostrarli. In questo modo ogni parte si può riusare e correggere da sola.**
- **Un menu interattivo si costruisce con un ciclo while True: che legge la scelta con input() e usa if / elif / else. break esce dal ciclo, continue torna all'inizio.**
- **L'input dell'utente può essere sbagliato: int(input(...)) genera un ValueError se non si scrive un numero, quindi va protetto con try / except.**
- **Per cercare un testo parziale si usa LIKE con i caratteri %, che vanno messi nel valore passato al segnaposto: ("%" + testo + "%",), non dentro la stringa SQL.**

Organizzare il codice: funzioni e menu con sqlite3

- **Come si compone il file: tutto sta in un unico file .py, con le parti in quest'ordine:**
  - **1. import sqlite3**
  - **2. funzioni del database: connetti(), crea_tabella(), aggiungi_pianta(), elenco_piante() ...**
  - **3. funzioni che stampano: stampa()**
  - **4. il menu: menu()**
  - **5. in fondo, le righe che avviano il programma: crea_tabella() e menu()**

- **Le funzioni devono essere definite prima delle righe che avviano il programma, perché Python le legge dall'alto verso il basso.**
- **Gli esempi non si riscrivono da capo: l'Intermedio aggiunge stampa() e menu() al file del Base (al posto delle righe di prova), l'Avanzato aggiunge due funzioni e sostituisce menu() con la versione completa.**

Esempio

Cosa mostra: dividere il codice in funzioni che aprono la connessione, eseguono un comando e restituiscono i dati.

```
import sqlite3

def connetti():
    conn = sqlite3.connect("giardino.db")
    conn.row_factory = sqlite3.Row  # righe leggibili per nome di colonna
    return conn

def crea_tabella():
    conn = connetti()
    with conn:  # commit automatico
        conn.execute("""
            CREATE TABLE IF NOT EXISTS piante (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                nome TEXT NOT NULL,
                luogo TEXT
            )
        """)
    conn.close()

def aggiungi_pianta(nome, luogo):
    conn = connetti()
    with conn:
        conn.execute("INSERT INTO piante (nome, luogo) VALUES (?, ?)", (nome, luogo))
    conn.close()

def elenco_piante():
    conn = connetti()
    righe = conn.execute("SELECT id, nome, luogo FROM piante ORDER BY nome").fetchall()
    conn.close()
    return righe  # la funzione restituisce i dati, non li stampa

# prova rapida (se rilanci il file, il basilico viene aggiunto di nuovo)
crea_tabella()
aggiungi_pianta("Basilico", "balcone")
for p in elenco_piante():
    print(p["id"], p["nome"], "-", p["luogo"])
```

- **connetti() è l'unico punto in cui compare il nome del file: cambiarlo è questione di una riga.**
- **Ogni funzione ha un solo compito e un nome che lo descrive.**
- **elenco_piante() fa return delle righe: chi la chiama decide come mostrarle.**
- **Le righe di prova in fondo si tolgono quando si passa all'esempio successivo.**

Esempio

Cosa mostra: un menu interattivo con while True, che richiama le funzioni dell'esempio Base.

```
# in fondo al file dell'esempio Base, al posto delle righe di prova

def stampa(piante):  # funzione che si occupa SOLO di mostrare i dati
    if not piante:
        print("Nessun risultato")
    for p in piante:
        print(p["id"], p["nome"], "-", p["luogo"])

def menu():
    while True:  # si ripete finché l'utente non sceglie "0"
        print("\n1) Aggiungi pianta   2) Mostra elenco   0) Esci")
        scelta = input("Scelta: ")
        if scelta == "1":
            nome = input("Nome: ")
            luogo = input("Dove si trova: ")
            aggiungi_pianta(nome, luogo)
            print("Pianta aggiunta!")
        elif scelta == "2":
            stampa(elenco_piante())
        elif scelta == "0":
            print("Arrivederci!")
            break  # esce dal ciclo e il programma termina
        else:
            print("Scelta non valida")

crea_tabella()
menu()
```

- **Il menu non contiene SQL: chiama le funzioni del database e stampa().**
- **break interrompe il while True quando si sceglie 0.**
- **L'else finale gestisce qualsiasi scelta non prevista, senza errori.**
- **stampa() si può riusare con qualsiasi elenco di piante (anche quello di una ricerca).**

Esempio

Cosa mostra: ricerca con LIKE, cancellazione con rowcount e controllo dell'input numerico.

```
# funzioni da aggiungere al file, e menu() da sostituire con questa versione

def cerca_piante(testo):
    conn = connetti()
    righe = conn.execute(
        "SELECT id, nome, luogo FROM piante WHERE nome LIKE ?",
        ("%" + testo + "%",)  # i % vanno nel valore, non nella stringa SQL
    ).fetchall()
    conn.close()
    return righe

def elimina_pianta(id_pianta):
    conn = connetti()
    with conn:
        cur = conn.execute("DELETE FROM piante WHERE id = ?", (id_pianta,))
        quante = cur.rowcount  # quante righe sono state cancellate
    conn.close()
    return quante

def menu():
    while True:
        print("\n1) Aggiungi   2) Elenco   3) Cerca   4) Elimina   0) Esci")
        scelta = input("Scelta: ")
        if scelta == "1":
            aggiungi_pianta(input("Nome: "), input("Dove si trova: "))
            print("Pianta aggiunta!")
        elif scelta == "2":
            stampa(elenco_piante())
        elif scelta == "3":
            stampa(cerca_piante(input("Testo da cercare: ")))
        elif scelta == "4":
            try:
                id_pianta = int(input("Id da eliminare: "))  # ValueError se non è un numero
            except ValueError:
                print("Devi scrivere un numero")
                continue  # torna all'inizio del while
            if elimina_pianta(id_pianta) == 0:
                print("Nessuna pianta con questo id")
            else:
                print("Pianta eliminata")
        elif scelta == "0":
            break
        else:
            print("Scelta non valida")

menu()
```

- **LIKE con %testo% trova tutte le piante il cui nome contiene il testo cercato.**
- **elimina_pianta() restituisce rowcount: il menu decide quale messaggio mostrare.**
- **try / except ValueError evita che una lettera al posto del numero faccia crashare il programma.**
- **continue salta il resto del ciclo e ripresenta il menu.**

Esempio

Variante – funzioni che gestiscono gli errori Cosa mostra: aggiungi_pianta() gestisce sqlite3.Error e comunica l'esito a chi la chiama.

```
def aggiungi_pianta(nome, luogo):
    conn = connetti()
    try:
        with conn:
            conn.execute("INSERT INTO piante (nome, luogo) VALUES (?, ?)", (nome, luogo))
        return True   # tutto ok
    except sqlite3.Error:
        return False  # qualcosa è andato storto: lo comunico a chi chiama la funzione
    finally:
        conn.close()

# nel menu, la voce "1" usa il risultato:
#   if aggiungi_pianta(nome, luogo):
#       print("Pianta aggiunta!")
#   else:
#       print("Non è stato possibile aggiungere la pianta")

crea_tabella()
print(aggiungi_pianta("Salvia", "orto"))  # True
print(aggiungi_pianta(None, "orto"))      # False: il nome è NOT NULL → IntegrityError
```

- **Il try / except sta dentro la funzione: chi la usa non deve preoccuparsene.**
- **return True / return False comunica l'esito senza stampare nulla nella funzione: il messaggio lo sceglie il menu.**
- **finally chiude la connessione in tutti i casi, anche dopo un errore.**
- **Lo stesso schema vale per le altre funzioni: in caso di errore restituiscono un valore che lo segnala (False, None o una lista vuota).**

Esercizi

- **Es1**
  - **Obiettivo:** nel database collezione.db gestisci una tabella giochi (id, titolo, anno) con tre funzioni: crea_tabella(), aggiungi_gioco(titolo, anno) ed elenco_giochi(), che restituisce le righe invece di stamparle. Nel programma principale aggiungi 3 giochi e stampa l'elenco.

- **Es2**
  - **Obiettivo:** nel database ricettario.db crea una tabella ricette (id, nome, ingredienti) e un menu con while True con quattro voci: aggiungi ricetta, mostra tutte le ricette, cerca per ingrediente (usando LIKE), esci. Il programma deve rifiutare un nome o un elenco di ingredienti lasciato vuoto, chiedendo di nuovo il dato.

- **Es3**
  - **Obiettivo:** nel database agenda.db costruisci una piccola agenda da terminale con la tabella appuntamenti (id, data, titolo, luogo). Il menu deve permettere di aggiungere un appuntamento, mostrare tutti gli appuntamenti in ordine di data, mostrare quelli di un giorno scelto, modificare il titolo di un appuntamento (indicando l'id), eliminare un appuntamento e uscire. Le funzioni che usano il database devono restituire i dati e gestire sqlite3.Error, l'id inserito dall'utente deve essere controllato (numero valido, appuntamento esistente) e le funzioni che stampano devono restare separate da quelle del database.



Disattiva video
Sicurezza
Partecipanti 9
Chat 1
Condividi schermo
Registra
Sessioni secondarie
Reazioni
App
Lavagne
Termina



Video
Condividi
9
Partecipanti
Chat
Reagisci
Altro
Cerca


Corrado Tani


Michela Cattabriga


mirko


Chiara Galazzo


fatjon itaj


Alessandro Rocco


Beatrice Gobbo


Lucia DenuzzO


sonia desimone


Sofia Mongiovetto